catanrl v2

experiment log

living document · last updated 17 october 2026 · github.com/mcrco/catanrl-v2 · wandb: catanrl-v2

the goal of this repo is a recipe for the best catan bot out there, for any number of players on any map, with every experiment a config file under `configs/experiments/` and every number below traceable to an eval record under `runs/`. v1 (catanrl) got a two-player bot to about 90% against catanatron's `ValueFunctionPlayer` (F); v2 rewrote the stack around a C++ engine and pufferlib and is past 95% in 2p from scratch. this log is organized by question: what architecture, what the current best recipe is and what it measures, and the list of ideas i'm working through (each with a checkbox).

1 basic arch ablations

previously did some informal arch ablations, and realized that the graph network proposed in canopy is the fastest, most efficient, and most generalizable (can scale to arbitrary maps since you predict board actions per intersection/road/tile). but would still be interested in doing a formal ablation with MLP, LSTM, CNN (like in the old catanrl), and GNN (canopy) eventually.

what the informal ablations (v1, branch `arch-ablation`) actually showed, so i don't have to redo them:

- on a teacher-neutral target (predicting the terminal outcome of held-out games, win/draw/loss cross-entropy in nats, 0.693 is a coin flip) the ranking was graph < MLP < CNN: `graph-s` 0.396 (0.5M params), `mlp-512` 0.464 (2.4M), xdim CNN 0.83 (16.8M). the CNN predicted the winner worse than a coin flip and its value head memorized within a few hundred steps. a graph transformer over the same node and tile tokens (`gtx-s`, 0.397) tied message passing, so what matters is the structured head (per-node settlement/city logits, per-edge road logits, per-tile robber logits), not the exact mixing layer.
- imitation loss is biased toward the teacher's family (search targets carry the teacher's logit shape), so architectures can't be compared on it. that is why the outcome target was used.
- under the full RL recipe (DAgger, then MARL PPO) a 78.5M-parameter CNN replica trailed graph-s by 7–12 points against F at a quarter of the training speed, and graph-m tied graph-s head to head (50.6%) at 2.3× the cost per step. step cost at batch 512: MLP 3 ms, graph-s 26 ms, CNN 120 ms; the graph net is launch-bound (about 200 small kernels per forward), which CUDA graphs fix at inference.

what the formal ablation should look like in v2. the harness already sweeps `model.preset` (`configs/experiments/arch-lineage.yaml` sweeps `graph-s`, `graph-m` and `mlp` over two seeds, but at 400M steps a cell is 6.5 h, so 12 cells is a week of GPU). the v2 row is the engine's padded seat-relative graph row, so the MLP is just a flat network over it (`configs/models/mlp.yaml`, padded dims only) and the CNN has to lay the BASE board out on a grid itself from the tile→node topology the row carries. an LSTM is the odd one out: it changes what the policy sees (a history instead of the latest state), so it is also the first step of the "how markov" idea in Section 3.

- ☑ port v1′s xdim CNN backbone to the v2 row (`src/catanrl/models/cnn.py`, preset `cnn`), BASE-map only like the MLP.
  - done 2026-10-17: v1′s `xdim-default` size (16.8M parameters per network, 15.1M of them in the projection after the convolutions), the tile/node/edge blocks gathered onto v1′s 11 × 21 plane that is laid out once from the engine's BASE map. deviations from v1, all documented in the module: GroupNorm instead of BatchNorm (so rollouts and evals run the same network; `norm: batch` restores v1), every block feature is a channel (47 at 2p) instead of v1′s 20 hand-picked planes, v2′s structured heads. a map with BASE's dims but another tile order (the `hexagon` generator) is refused at launch rather than silently scrambled, which a review caught.
- ☑ an LSTM policy over each seat's row history through pufferlib's `use_rnn` path (preset `lstm`): the trunk embedding feeds an LSTM cell whose state persists across env steps and resets at game ends.
  - done 2026-10-17: the `graph-s` trunk, an LSTM cell over its pooled embedding stepped at each of the seat's own decisions (waiting rows keep their state, a game's first decision starts from zero), the memory injected into both heads; 0.9M parameters. PPO replays the recurrence over segments × horizon from the stored rollout states (CleanRL's PPO-LSTM shape); a 5-epoch run at learning rate 0 reproduces the rollout's logits

exactly (KL 0, clip fraction 0). the native `POLICY` agent keeps one memory stream per seat of a game, so eval records don't depend on `num_envs`; MCTS, BELIEF and the server bridge refuse it for now (a sampled world has no decision history of its own).

- ☑ `configs/experiments/arch-ablation.yaml`: the current best recipe (Section 2) at the fair `private` level, one seed, 100M agent steps per architecture (about 1.6 h each for the graph nets and the mlp, the cnn slower; puffer-graph2 was at 86% vs F by then, and v1′s gaps were 7–12 points, so a 100M-step cell separates them), selected on `select-2p`, reported on `heldout-2p` vs F and head to head against the graph net on the same boards. written and validated 2026-10-17; every cell is `shared: true` except the lstm, which has one trunk by construction.
- ☑ run it. ran 2026-10-18, 01:37 to 13:08, one invocation (`catanrl run arch-ablation`; the runner orders the cells because the head-to-head rows depend on `graph-s`).

1.1 what the formal ablation says

same recipe, same 100M agent steps, same boards, one seed, `private` level. "vs F" is the argmax policy on `heldout-2p` (5,000 games); "vs graph-s" is the same policy against the `graph-s` cell's best checkpoint on the same boards.

Table 1: the formal ablation (`configs/experiments/arch-ablation.yaml`, records `039b60f0`, `1cd2de87/073da498`, `76c88ebd/189e5e8d`, `8e2e23d8/5abf6f0c`). the aux row is `configs/experiments/aux-hidden.yaml` (`0e28d5cf/c6b7f682`), the same `graph-s` cell plus the hidden-feature head of Section 3. wall time includes the select evals; the mlp's 0.3 h is why it is the cheap baseline and the cnn's 7.5 h is why v1 abandoned it. the 4p rows are `configs/experiments/aux-hidden-4p.yaml` (`d97f69b3`, `da4cb5be`; head to head `27a774b6/2eb6a290`, one table with two F in both slot orders, 8,000 games) and `configs/experiments/lstm-4p.yaml` (`9198cef1`; head to head against the plain arm `5efc75ba/cf8368f4`, the same table) and `lstm-aux-4p.yaml` (`722b7168`; against the lstm `561c7ee2/f691b7fc`), 750 epochs = 49M decisions, the first runs under the lambda critic default. live logs: the `arch-ablation`, `aux-hidden-4p`, `lstm-4p` and `lstm-aux-4p` groups on wandb.

arch	params	steps/s	wall	select-2p (best)	vs F	vs graph-s
`graph-s` (shared trunk)	0.54M	17.6k	1.6 h	92.5 (e750)	**90.7** 89.9–91.5	—
`lstm` (graph-s trunk + LSTM)	0.86M	14.8k	1.9 h	92.0 (e700)	**90.3** 89.4–91.1	**51.0** 49.7–52.4
`graph-s` + aux head (Section 3)	0.62M	17.1k	1.7 h	90.0 (e700)	89.1 88.2–89.9	47.7 46.3–49.1
`cnn` (v1′s xdim)	16.8M	5.6k	7.5 h	81.5 (e700)	80.4 79.3–81.5	33.7 32.4–35.0
`mlp`	2.2M	108k	0.3 h	68.0 (e700)	64.9 63.6–66.2	18.1 17.0–19.2
4 players, 750 epochs (`aux-hidden-4p`: `ppo-4p`'s recipe at the 2p cells' budget per decision; select-4p and vs F are shares at a table of three F, parity 25)						
`graph-s` plain (the lambda critic default)	—	30.5k	1.85 h	65.8 (e600)	64.0 62.5–65.5	—
`graph-s` + aux head	—	29.6k	1.92 h	68.5 (e700)	63.2 61.6–64.6	42.5 vs plain's 42.5 with two F at the table, paired 0.0 pp −2.0–1.9
`lstm` (graph-s trunk + LSTM; `lstm-4p`)	—	22.3k	2.53 h	65.2 (e750)	65.5 64.0–67.0	45.2 vs plain's 40.8 with two F at the table, paired +4.4 pp 2.4–6.3
`lstm` + aux head (`lstm-aux-4p`)	—	21.8k	2.58 h	68.4 (e750)	**66.2** 64.7–67.6	43.8 vs the lstm's 42.3, paired +1.5 pp −0.4–3.5

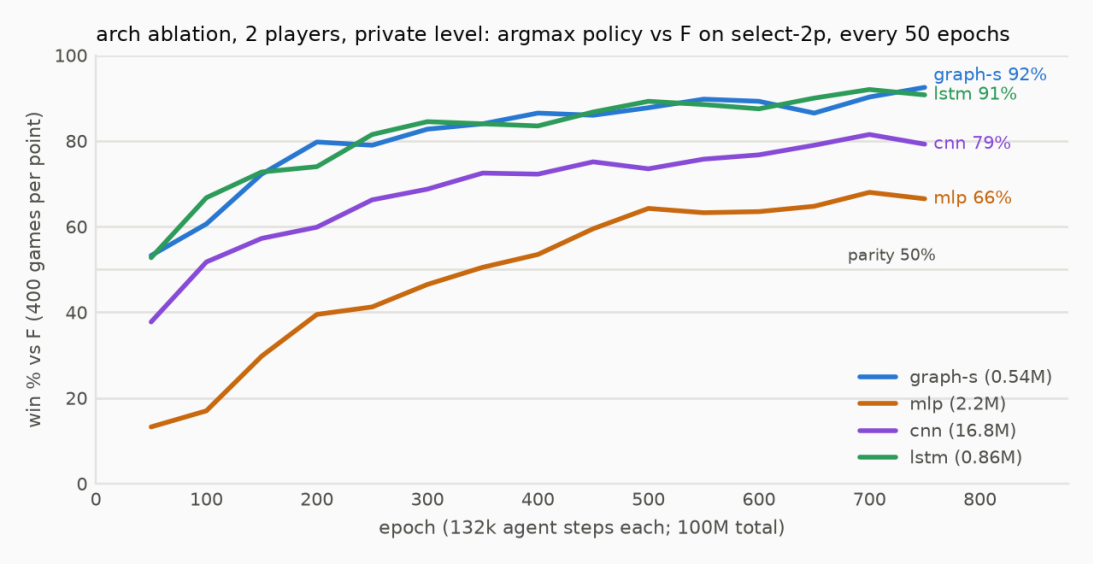


Figure 1: the four cells' selection curves on the same axis (argmax policy vs F on `select-2p`, every 50 epochs). the graph net and the lstm are indistinguishable from epoch 300 on; the cnn tracks ten points lower; the mlp never catches up.

so, with the informal ablation now formal: graph ≈ lstm > cnn > mlp, and the structured head is what matters. the lstm, which is the graph net with memory over the seat's own decisions, ties it head to head (51.0, a coin flip) at 1.6× the parameters and 0.85× the throughput, so at this budget a recurrent state adds nothing the current row doesn't already carry (more on this under the "how markov" idea in Section 3). the cnn, 31× the parameters and a third of the speed, is ten points behind against F and loses two of three games to the graph net; the mlp is 26 points behind and loses four of five. one seed, 100M steps, 2 players: enough to rank the families, not to separate graph from lstm, which would need the full 500M-step budget and more seeds.

2 current best recipe

implement engine in c, turn into pufferlib env, run for 1B steps, and get a really good shared-trunk critic + policy that beats `ValueFunctionPlayer` 90% of the time.

put the critic + policy inside a mcts harness, with a simple probabilistic belief-sampling determinization method to get another 5% h2h edge over the raw policy.

2.1 the recipe, concretely

- **engine.** cppanatron, a C++20 port of catanatron kept in lockstep parity with it (2–6 players, explicit maps of any shape). the observation encoder, the action codec, the threaded multi-game env, the hand tracker and the batched MCTS all live in C++, so training, search and deployment can't drift apart. one float32 row per (game, seat), seat-relative, padded to the env's maxima: `[global | players | tiles | nodes | edges | topology | action mask]`.
- **network.** `graph-s`: canopy nexus-v3 style heterogeneous message passing (node↔node, node↔tile, node↔edge), hidden 96, 3 layers, one shared trunk for the policy head and a per-seat value head (one sigmoid per seat = that seat's win probability; the acting row's prediction fills the waiting seats' values, so GAE on each seat's own trajectory costs one network call per game step). 541,540 parameters.
- **training.** pufferlib 3 PPO with two small patches (losses over acting rows only, a value-loss hook), pure self-play (every seat learns), muon at 5e-3 with a cosine to 5e-4, momentum 0.95, γ = 0.999 per decision, GAE 0.95, 2 update epochs, clip 0.2, entropy 0.005, prioritized 64-step segments, fp32. the batch shape is v1′s graph2 run exactly: 1032 games × 2 rows × 64 steps = 132,096 agent steps per epoch, 16 optimizer steps per epoch, 3786 epochs. `configs/experiments/puffer-graph2.yaml` lists every deviation from v1 and why.
- **search.** `MCTS:ckpt=…:sims=N:dets=K`: a batched N-player PUCT search (per-seat values backed up max^n, exact chance nodes for dice, dev-card draws and steals) over K worlds sampled from the hand tracker's belief at the root (`dets=K`; `oracle=true` searches the true game and is only a reference). `BELIEF:…:worlds=16` is the raw policy averaged over 16 sampled worlds, the fair way to play a network that was trained on more than its seat sees.

2.2 how things are measured

every number is on a held-out suite of boards that training never saw (`validate` checks the training pools against them): `heldout-2p` is 2,500 boards (seed 67) played in both seat orders = 5,000 games; `heldout-4p` is 1,000 boards (seed 68) × 4 rotations = 4,000 games. checkpoints are chosen on a disjoint `select-*` suite every 50 epochs, so the held-out number is never the one that picked the checkpoint. intervals are wilson 95%. in N-player games the number is slot 0's *score share* (a win counts 1, a truncated game 1/N to every seat), so parity is 1/N. agents that use information a player doesn't have (a `full` policy, an oracle search) are marked; everything else is fair.

2.3 results so far

Table 2: the current best agents against F. v2 trains from scratch (no DAgger) and lands 5 points above v1's warm-started reference in 2p. the 2p `full` policy sees F's development cards; its belief-averaged version doesn't and loses nothing (95.06 vs 95.16), which is the same "hidden information is worth under a point" finding as v1. in 4p the shared-trunk network is trained and played at the `private` level, which is the only fair level past two players.

agent	sees	games	vs F	record
2 players, BASE, 15 vp (`puffer-graph2/full@e3350`, 500M agent steps, 7.95 h)				
raw policy, `full` obs	hidden cards	5,000	**95.16** 94.53–95.72	`ff8528a4`
raw policy, belief-averaged (`BELIEF`, 16 worlds)	fair	5,000	**95.06** 94.42–95.63	`67869e7a`
raw policy at the `public` level (`puffer-graph2/public@e3600`, same recipe, 7.95 h; no belief averaging needed)	fair	5,000	**94.96** 94.3–95.5	`dc8a611a`
MCTS 64 sims on the true game	hidden cards	500	94.8 92.5–96.4	`3fdae985`
MCTS 256 sims on the true game	hidden cards	500	95.2 93.0–96.8	`609e0aa7`
MCTS 1024 sims, 1 det., vs `BELIEF` 16 worlds (head to head)	fair	1,000	53.4 50.3–56.5	`04fd2c25`
MCTS 4096 sims, 1 det., vs `BELIEF` 16 worlds (head to head)	fair	500	55.2 50.8–59.5	`2aaf2eb7`
MCTS 1024 sims, 1 det., vs the raw `public` policy (`puffer-graph2/public@e3600`, head to head)	fair	1,000	54.9 51.8–58.0	`72819d1a`
MCTS 4096 sims, 1 det., vs the raw `public` policy (head to head)	fair	500	55.2 50.8–59.5	`0ef1b81e`
raw policy on one predicted row (`PREDICT`, `aux-hidden` predictor, Section 3)	fair	5,000	94.72 94.1–95.3	`b74d240b`
`PREDICT` vs `BELIEF` 16 worlds (head to head)	fair	5,000	50.4 49.1–51.8	`e7e34d28`
4 players, BASE, 10 vp (`ppo-4p/main@e3650`, 1B agent steps = 249M decisions, 9.3 h)				
raw policy vs F, F, F (score share, parity 25%)	fair	4,000	**78.55** 77.25–79.79	`e5b302ee`
two copies vs F, F (team share 93.2 = 40.45 + 52.75; the split is the slot order, Section 3)	fair	4,000	93.2	`b07e5bd9`
raw policy vs R, R, R	fair	4,000	100.0 99.9–100.0	`95570c74`
MCTS 1024 sims, 1 det., vs three raw copies (critic-fix net `lambda-critic-g1@e100`; share, parity 25%)	fair	2,000	26.9 25.0–28.9	`657b314d`
v1 reference (graph-s, DAgger + MARL PPO, 73.7M steps), for history				
raw policy, belief-averaged	fair	2,000	89.65 88.24–90.91	v1 report
MCTS 256, determinized	fair	1,000	90.5 88.5–92.2	v1 report
MCTS 1024 on the true game	hidden cards	1,000	92.7 90.9–94.2	v1 report

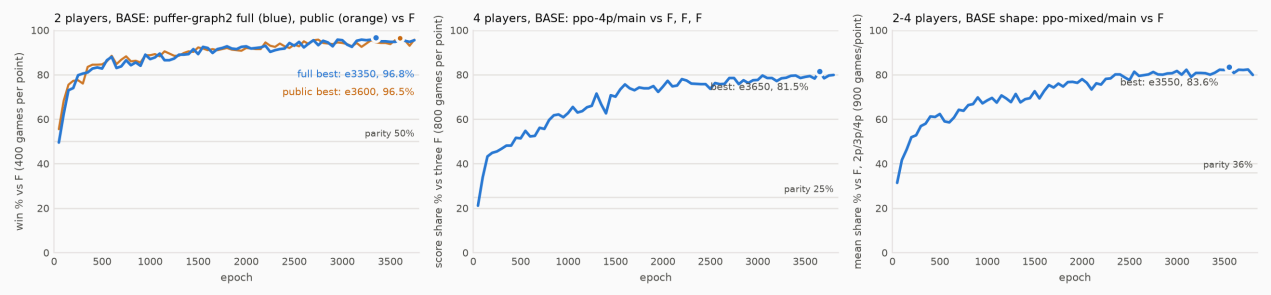


Figure 2: training-time selection curves (argmax policy vs F on the `select-*` suites, every 50 epochs). all runs keep improving through the second half of the cosine, so matched comparisons need the whole budget; the 2p `public` variant (orange) tracks `full` (blue) throughout. full per-epoch logs: puffer-graph2/full, puffer-graph2/public, ppo-4p/main and ppo-mixed/main on wandb.

the wandb runs were backfilled from `runs/*/metrics.jsonl` and the eval records after the fact (the trainer writes jsonl; wandb mirrors and never decides), so they hold everything the trainer logged: losses, entropy, KL, clip fraction, game length, seat win rates, the critic's calibration (`critic/value_sum_*`: the per-seat win probabilities should sum to about the share of games somebody wins), throughput and the select curve. `ppo-4p`'s critic

summed to about 1.2 over its four seats against a realized 0.88, a bias from V-trace targets on waiting rows; `configs/experiments/ppo-4p-critic-fix.yaml` fixed it on 2026-10-18 (below). since 2026-10-17 there is a live mirror (`catanrl.wandb_mirror`, on whenever an experiment's `meta.wandb.project` is set; the record on disk stays the source of truth and every wandb failure is a warning) and `tools/wandb_backfill.py` re-syncs any run directory into the same wandb run, so a run launched without the mirror, like `ppo-mixed`, lands there too.

2.4 what is still missing for this section

- ☑ **the 5% search edge in v2, head to head.** v1 measured MCTS against the same network's belief-averaged raw policy on 1,000 games: 51.1% at 256 sims, 55.1% at 1024, 58.1% at 4096 (against F both are near the ceiling, which is why the vs-F rows above don't move). v2, same network, fair search (`dets=1`, worlds from the hand tracker) against `BELIEF` 16 worlds on `heldout-2p`, finished 2026-10-18: **53.4** 50.3–56.5 at 1024 sims (n=1,000, `04fd2c25`) and **55.2** 50.8–59.5 at 4096 (n=500, `2aaf2eb7`). so the edge is there and grows with sims, a point or two smaller than v1′s over a stronger raw policy; both seats of the pairing agree (first seat 55.0 / second 51.8 at 1024). cost: 11.5 h of gpu for the 1,500 games at 128 games in flight, which is what parks alphazero at these sim counts (Section 3). the `public` network's own rows (next item) repeat it over its raw policy: 54.9 at 1024, 55.2 at 4096, in 5.0 h.
- ☑ **the fair 2p variant.** `puffer-graph2/public` (the `public` level: in 2p every resource movement is seen by both seats, so the opponent's exact hand is a perfect card counter's knowledge, and a `public` policy is fair without belief averaging; what `full` adds over it is only the opponent's dev cards in hand) ran 2026-10-18 18:03 to 2026-10-19 02:06: the same recipe and budget as `full`, the same 7.95 h at 17.8k steps/s, best checkpoint e3600 (select 96.5 against `full`'s e3350 at 96.8; the two select curves lie on top of each other, Figure 2, mean over epochs ≥ 3000 94.4 vs 95.1). final: **94.96** 94.3–95.5 vs F on `heldout-2p` as a plain fair `POLICY` (`dc8a611a`, 5,000 games), against the `full` variant's belief-averaged 95.06 94.4–95.6 and its oracle 95.16, so the one thing `public` cannot see, the opponent's dev cards, is worth nothing measurable in 2p (the aux head found the same from the other side: the hand is inferable, the dev cards are not, and the policy did not need them). one forward pass, no sampled worlds, no tracker: this is the bot to serve raw. its own search rows ran 02:07 to 07:10 (5.0 h for the 1,500 games, half of what the `full` finals took: the opponent here is one forward pass, not sixteen, and 256 games were in flight instead of 128): MCTS 1024 sims, one world from the tracker, beats the raw `public` policy **54.9** 51.8–58.0 on 1,000 games (`72819d1a`, 1.5 h) and 4096 sims **55.2** 50.8–59.5 on 500 (`0ef1b81e`, 3.6 h), the same edge the `full` network has over its belief-averaged self (53.4 / 55.2 above). both seats agree (the searcher 60.6 in seat 0 and 49.2 in seat 1 at 1024, the raw policy 50.8 / 39.4), with a first-seat advantage under both: seat 0 wins about 56% of these games whoever sits there.
- ☑ **the critic fix.** `ppo-4p`'s critic summed its four seats' win probabilities above the realized sum (about 1.2 against 0.87 at epoch 300; 0.93 against 0.90 by the end), from two causes found on cpu replications: V-trace ratios on the waiting rows of each sample (only the acting seat's row gets min(ρ, 1), the other three get 1, so each seat's target values the game as if that seat alone played the improved policy, and the four one-sided gains compound through bootstrapping) and muon's fixed-size update of the 1 × 128 value output layer (the rollout network lands on the high side every epoch). `configs/experiments/ppo-4p-critic-fix.yaml`, run 2026-10-18 (four arms × 15 min): 100-epoch fine-tunes of `ppo-4p/main@latest` at its final learning rate with lambda-return critic targets (every ratio 1; the policy's advantages stay V-trace's), at three discounts and one with the output layer on AdamW. every arm puts the seat sum on the realized sum within about 20 epochs and keeps it there (Figure 3): `lambda` 0.906 against 0.899 realized at γ = 0.999 (from 0.920 against 0.914 in the first epochs), `lambda-critic-g1` 0.998 against 1.000 with the critic undiscounted, `lambda-g1` 1.000 against 1.000 with policy and critic undiscounted, `lambda-adamw` 0.906 against 0.898. the rollout-vs-trained gap falls from 0.004 to 0.000 in all four, so the targets alone remove muon's phase lock and the AdamW arm buys nothing. per-slot calibration at the end of `lambda-critic-g1`: the seat to move predicted 0.284 against a realized 0.276, the other three within 0.004. the policy does not move in 100 epochs: select-4p 79.9 at the start and 80.4 / 79.9 / 80.4 / 79.1 at the end of the four arms (n=800 per point, ±2.8), clip fraction, entropy and game length (79–80 turns) unchanged, and the undiscounted policy plays games of the same length, so γ = 1 changes nothing the policy optimises at this horizon. recommendation, applied to the defaults on 2026-10-19: `critic_targets: lambda` and `critic_gamma: 1` in `configs/train/ppo.yaml` for every run from now on (per-seat win probabilities that sum to one are what max^n backups want); the experiment yamls with stored runs pin the previous values, so their config hashes and their records are unchanged. from scratch (2026-10-19, the two `aux-hidden-4p` arms, the

first runs under the new default; Figure 3, right): the undiscounted lambda critic overshoots early, a seat sum of 1.83 at epoch 21, 1.39 at 50, 1.31 at 100, 1.18 at 200, then lands on the realized 1.00 by epoch 220, where the learning rate is still four fifths of its peak, and holds a mean of 1.000 over epochs 300–750, within 0.06 at every epoch (the aux arm the same, mean 0.99). `ppo-4p/main`'s V-trace critic on the same recipe read 1.27 at epoch 100, 1.21 at 300, 1.09 at 500, 1.06 at 750 and never got there (0.94 at epoch 3800 against a realized 0.90). so the early overshoot is a from-scratch transient the fine-tunes could not show (their critics started trained) and both target rules produce it; only the lambda critic recovers from it. explained variance 0.82–0.85 at the end, as before. live logs: the `ppo-4p-critic-fix` group on wandb.

- ☑️ **the 4p search rows.** `configs/experiments/search-4p.yaml`, run 2026-10-18 (71 min for 400 games): `MCTS 1024 dets=1` with the critic-fix network (`ppo-4p-critic-fix/lambda-critic-g1@e100`, undiscounted per-seat values that sum to one, the same raw policy as `ppo-4p` to within noise) against three copies of that network's raw policy on `heldout-4p`, 100 boards × 4 rotations: a 27.0 22.9–31.6 share (`90a1f06c`) against the raw copies' 24.3 and a parity of 25. so the search edge in 4p is about two points of share (8% relative), the same order as the 2p edge's 3.4 points of win rate, and 400 games cannot separate it from zero. `configs/experiments/search-4p-2000.yaml`, the same table on 500 boards × 4 rotations, ran 2026-10-19 07:38 to 10:15 (2.6 h for 2,000 games with 512 in flight, 4.7 s a game; the first 100 boards replay the record above, 400 of its 400 games identical, and the other 1,600 alone give 26.9): **26.9** 25.0–28.9 (`657b314d`, 538 wins of 2,000) against the raw copies' 25.2 / 22.7 / 25.3 (24.4 pooled). the point estimate did not move and the interval now ends on parity (p = 0.053 against 25% by the record's own test), and the searcher is ahead of the raw copies' mean at every seat (+3.4 in seat 0, +2.1, +3.3 and +1.3 in seats 1–3). read it as a two-point edge, 8% relative, measured to about ±2: the same relative edge as 2p's (7–10% there), real but small at 1024 sims and one world, and a long way from paying for its cost (each searcher decision is 1024 forwards). the first seat wins 27.9% of these games overall against seat 2′s 23.1, so a small first-seat advantage sits in these numbers too (no F at the table, so no F targeting; the four rotations are summed).
- ☑️ **tree reuse and leaf deduplication in the search** (marco's ask, 2026-10-20, for the deployed bot). the engine's pool could re-root a search at a child (`advance`) but the agent never asked, and chance outcomes of one node were merged while positions reached by different action orders were not. built in a worktree the same night: `MCTS:...:reuse=true` keeps a seat's search alive across its decisions and re-roots it when the game advanced by exactly its own action (a fresh simulation budget on top of the retained visits; about a third of a seat's decisions in 4p and half in 2p re-root, the rest follow a roll, another seat's move or a forced move); `transpositions=true` gives the pool a per-world transposition table (positions reached by different build, buy or discard orders share one node with shared statistics; the discard chain after a 7 collapses from a tree to a lattice of at most fifty nodes against 86; links per search from a handful to over a hundred by position; the simulation count is identical and the search deeper rather than smaller); and the search driver evaluates byte-identical rows once (bitwise neutral under the strict evaluator, so it has no key; rows forwarded to the network fall to 46–91% depending on worlds and players, by a cache of rows seen in earlier rounds). both options are keyed only when set, so every stored search record rebuilds unchanged. the comparison rows against the stored search tables (`mcts-reuse-2p`, `mcts-reuse-4p-both`, `mcts-reuse-4p`, paired on shared game ids with `tools/search_probe.py --pair`) ran after the trading rows. 2p (`mcts-reuse-2p`, 04:24–05:09, 45 min for 200 games): the searcher with both options takes **58.5** 51.6–65.1 of the games against the stored row's 59.5 on the same 200 game ids (`12f704f2` against `72819d1a`), paired −1.0 pp with 7 games flipping one way and 9 the other (p = 0.80): no regression, and mostly the same games, which is what a deeper search at the same simulation count looks like at this budget; the strength question is the follow-up row, equal share at half the sims. reuse on the server bridge, for the deployed bots, is merged (a seat's search survives its own moves; one fresh search in a recorded 1v1 took 40 s on a busy box against a 2 s median, under investigation); the openhex seat is being wired to it.

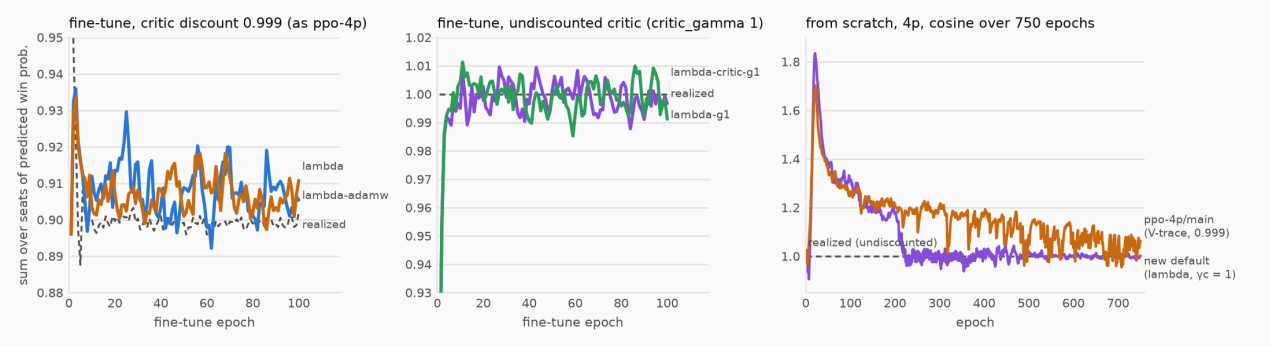


Figure 3: `ppo-4p-critic-fix`: the four seats' predicted win probabilities summed, per fine-tune epoch, against the realized sum (dashed: about 0.90 at γ = 0.999, 1 undiscounted). lambda-return targets put the sum on the truth in every arm within about 20 epochs; the AdamW output layer changes nothing. right: from scratch at 4p under the new default (`aux-hidden-4p/plain`, purple) against `ppo-4p/main`'s V-trace critic over its first 750 epochs (orange; its own realized sum is about 0.90 at γ = 0.999, the dashed 1.0 is the undiscounted critic's): both overshoot early, the lambda critic is on its realized sum from epoch 220, the V-trace critic is still 0.16 above its own at epoch 750.

3 exploring ideas

the list, with a box per idea; sub-bullets are the thinking so far and what has actually been done. running experiments are dated.

- ☐ **training alphazero** (currently agreement is really high, doesn't seem to yield many gains)
  - the loop exists (`train.algo: alphazero`, `configs/experiments/az-from-ppo.yaml`): self-play through the batched search with gumbel roots, a replay buffer of (row, visit-count policy, per-seat outcome), warm-started from a PPO checkpoint, judged head to head against the network it started from (against F a strong policy is at its ceiling, so vs-F can't see the gain).
  - the one run so far (2026-10-14, a 30-minute sanity run of `az-from-ppo`'s az stage in a scratch runs dir, since deleted; the numbers survive only in that session's transcript) started from a 40-minute ppo sanity network at 57.8% vs F, not from puffer-graph2: gumbel targets at 64 sims on a quarter of moves (16 on the rest), oracle search (`dets: 0`), pure self-play, lr 5e-5, 25 iterations. the raw policy fell to 46.6, 24.4 and then 18.5% vs F (600 games each), and nothing was measured head to head against its start. the bug hunt found one small bug (the gumbel temperature-0 finalist, fixed in cppanatron `5d9dda9`), which doesn't touch the targets. the policy wasn't better than its own search either: at e0 the 64-sim search scored 63.5% vs F (n=100) against the raw 57.8%. what it did find is noisy targets: with rescaled completed-q a near-single-visit q gap swings a target by 6.5 logits, two seeds agree on the top move 87% of the time (60% on discards), knight-before-roll went from 0.90 to 0.27, and fitting all that took the network's entropy from 0.42 to 0.82 until its argmax drifted to end-turn. whether ≥ 1024-sim targets fix it is untested: v1 never trained above 512 sims, and 512 didn't move an 87% network (v1′s frozen-teacher distillation of an 8 × 64-sim search, EX-1d, gave +1.3 vs F and 50.6% head to head, both null); the v2 search edge they would distill is now measured (53.4 at 1024, 55.2 at 4096, above).
  - **design (2026-10-21, CPU measurements, not run on the GPU yet):** does the search reuse above make self-play cheaper? barely. the tree survives 43% (2p) to 53% (4p) of self-play moves, but `advance` resets the budget, so today reuse saves no leaves at all; an engine top-up budget (search only `sims − kept visits`) would save 15–20% of them, and kept visits are worth about their count, so reuse can't halve the budget. transpositions buy nothing here and fight reuse (they restart the new root's shared children), so they stay off. the real lever is an eval cache across one self-play step: 31–46% fewer forwards (hashing has to move into the engine). a 1024-sim move costs 1,000 leaf rows and 80 ms of engine cpu; the first run as a config-only change of the existing trainer (1024-sim gumbel targets on every move, 512 games, from `puffer-graph2/public`) is 13–22 gpu hours for 60 iterations. caveats from the critique: the 2p probes ran the engine's default rules (10 vp, discard 7) instead of base-2p's (15, 9), so the 2p numbers need a rerun; the warm-start critic is a discounted v-trace critic while az trains undiscounted win/loss, so the value head drifts during the test; and an offline distillation gate (one fixed set of 1024-sim targets, several policy-only

fits judged head to head) would answer "is there anything to distill" more cheaply than the online loop. waiting on which network and which first step.

- **ran 2026-10-21 15:40–18:53, stopped at e8** (on wandb: az-2p-2048/main): marco's pick, the 2p specialist straight into the self-play loop at 2048-sim targets, engine work only after a positive result. `configs/experiments/az-2p-2048.yaml`: from `puffer-graph2/public@e3600` (actor and critic), base-2p, 512 games in flight, fair gumbel search (`dets: 1`, m = 16) on every move, 30 iterations × 16,384 targets, lr 5e-5; a head to head against the start every 2 iterations (4,000 games), raw finals on `heldout-2p` (vs the start, vs F, and best and start each vs the belief-averaged `puffer-graph2/full`, paired by board), the search finals only if the raw result is positive. new logging-only monitors (bit-identical training with and without them): the value head on finished games before it trains on them, how often the search overrules the network's argmax, and drift from the start on a fixed probe of 4,096 positions. the cpu probes under the right rules first: 2048-sim targets cut the prior's distance to an 8192-sim reference from 0.295 to 0.197 tv (the reference's own noise floor is 0.179) and change the top move at 20% of positions, but they're barely better than 1024 (−0.016 −0.034–0.001); robber moves and positions where the opponent holds a hidden vp card stay noisy. first iteration: 23.6k leaves/s, 24 min, so 12 h for the 30.
- **result: stopped at e8** (3.2 h) by the review's rule (two heads to head ≤ 48.5 in a row). the raw policy lost to its start: **48.0** at e6 and **48.2** at e8 (4,000 games each; the first update was in iteration 5, once enough games had finished), so the best stayed e0 and there are no finals to play. it went the way the 2026-10-14 run went, only slower: the network's entropy climbed (0.088 ⟶ 0.125), the search overruled its argmax more often (0.18 ⟶ 0.23, where a policy that learns from the search should make that rarer) and knight-before-roll fell from 0.53 to 0.43 on the drift probe. the critic wasn't it. the monitor's out-of-sample value loss rose from 0.20 to 0.54, but that's the games getting longer (the first games to finish are the short ones): the starting network on the same rows scores 0.375 / 0.508 / 0.387 at iterations 6–8, against the trained net's 0.359 / 0.538 / 0.402, so the critic moved by a few hundredths either way. the targets were it, as far as one look can say. gumbel's $\sigma(q) = (50 + \max \text{visits}) \cdot 0.1 \cdot q$, with q min-max rescaled to [0, 1] across the root's moves, puts a 100-logit spread between the best and the worst move at 2048 simulations, whatever the gap in win probability between them. so every one of the 16,153 two-move decisions in the replay (knight or roll, mostly) got a target of ≥ 0.99 on one move, and so did 81% of the decisions with more than eight legal moves. on the knight-or-roll decisions the start was ≥ 90% sure 98% of the time, the targets went against its argmax 21% of the time, and they play the knight 44% of the time against the start's 53%: the drift the network then learned. the moves behind those targets are good ones: `gumbel-edge-2p` (18:56–22:50, gumbel-edge-2p/main), the same 2048-sim gumbel search with the start's weights against its raw policy, wins **62.4** 59.4–65.3 of 1,000 games (`831eea84`), far above puct over the same policy (54.9 at 1024 sims, 55.2 at 4096). so the search had plenty to teach and the raw policy still got worse learning it; softer targets (no rescale, or a smaller scale) need the two constants exposed to the trainer. marco's point on the stop: the raw policy is the wrong thing to judge alphazero by, since the goal is a better network inside the search. the sound test is the trained network in the same search against the same raw start, paired game by game with the row above (mcnemar over the shared game ids, `tools/search_probe.py --pair`): `gumbel-edge-2p-e8` (22:51–02:59), e8 in the same 2048-sim gumbel search against the same raw start: **61.8** (`c4ea7b52`) against the start's weights' 62.4, paired over the 1,000 shared games **−0.6 pp** −3.7–2.5 (124 games won only by e8, 130 only by e0, mcnemar p = 0.75). only a quarter of the pairs differ at all (the board, the seat and the dice decide most games), so the pairing roughly halves the interval that two independent rows would give. the training didn't make a better search network either: the raw policy lost two points and the search played the same, so the stop didn't cost anything at e8, but by luck; the next run is judged in the harness.
- **ran 2026-10-22 15:49 to 2026-10-23 01:57, stopped by marco after e15** (on wandb: az-2p-2048-soft/main): marco's next step, softer targets with the search left alone. the trainer now rebuilds gumbel's improved policy itself from each search's visits and q and the network's outputs at the root (`train.policy_target: gumbel`; at the engine's constants it is the engine's target to 3e-8, and the default trains bit-identically to before). the scale was picked on cpu: 600 positions of the start's raw-policy base-2p self-play, two 2048-sim searches each, scored by −log of the target's mass on an 8192-sim search's move. a gentler σ alone softens nothing: the start's prior is itself one-hot (a median gap of 47 nats between its top two logits on knight-or-roll decisions), so softmax(prior + σ) stays one-hot at any scale, and no σ-only

setting scored apart from the engine's (the best −0.10 −0.24–0.03 nats). what does soften it is tempering the prior inside the target: softmax(logits / 10 + (50 + max visits) · 0.15 · q), q unrescaled (my proposal from the probe, approved by marco). it keeps the same first choice (agrees with the 8192 move 80.5% of the time against 81.4%) and spreads the rest where the deep search is unsure: −1.12 −1.39–−0.86 nats on the score, entropy 0.64 against 0.05, ≥ 0.99 on one move in 22% of decisions against 87%. that is calibration, not better moves: two 8192-sim searches pick the same move only 86% of the time, and the expected regret under the softer target is slightly worse (+0.05 0.01–0.08 win points). whether a network trained on it searches better is what the run tests; everything else is `az-2p-2048`'s (`configs/experiments/az-2p-2048-soft.yaml`). it is judged in the search harness throughout, as marco asked: every 5th iteration the checkpoint in a 512-sim gumbel search plays the raw start on the same 1,000 select games (e0′s weights are played too, so each select pairs with e0′s), and the finals put the best and the last checkpoint in the 2048-sim search against the raw start on `gumbel-edge-2p`'s games, each paired with `831eea84` by mcnemar (holm over the two). every iteration's weights are kept, so an e8 row can pair with `c4ea7b52` later. 19 h of training and selects, then 4–8 h of finals. stop rule: only a collapse, two selects in a row more than 4 pp below e0′s on the same games (about 2.5 paired sds); the raw policy is not a stop criterion this time. selects so far (512 sims, paired with e0 over the same 1,000 games): e0 57.4, e5 56.0 (−1.4 pp, 129 games won only by e0 against 115 only by e5, mcnemar p = 0.41), e10 53.8 (−3.6 pp, 158 against 122, p = 0.036), so the search network is getting worse, not better, though the rule hasn't tripped. the monitors say why: by e13 the network's entropy is 0.68 (0.09 at the start) and it agrees with the start's argmax on 72% of the drift probe, and the search overrules its argmax 41% of the time (18% at the start). the softer target is pulling the prior flat, as the fixed-point note predicted, faster than the search's q can replace what the prior knew. **result:** e15 in the same search lost to the raw start, **46.7** 43.6–49.8 (`8906dcf7`), −10.7 pp against e0 over the same 1,000 games (225 won only by e0, 118 only by e15, mcnemar $p < 10^{-6}$), with the network's entropy at 0.92 and the search's depth down from 11 to 8.2. the rule would have needed e20 too; marco stopped it ("it is sucking balls"). best stays e0, so there are no finals. the canopy comparison below puts the blame on the prior temperature, my proposal: canopy never tempers the policy the search reads.

- **what canopy did, and where we diverge** (marco asked, 2026-10-23; read from cullback/canopy's code and history and checked against source): canopy trained its 2.4M-parameter nexus-v3 net by pure self-play, most likely from random weights, on 1v1 colonist rules (balanced dice, friendly robber): gumbel alphazero (m = 6, 1600 sims on a quarter of moves and 64-sim value-only searches on the rest), the opponent's dev cards resampled every simulation, an improved-policy interior, a soft-policy side output (π^(1/4), weight 8) that the search never reads, a win/draw/loss value blended from the outcome toward the search's q (to 85% over 60 iterations), a 500k replay, adamw 5e-4 ⟶ 1e-4, about 315 iterations (47M positions, inferred from the checkpoint name). its evidence of success is thin: no win rates, elo or opponents are published, only a readme line ("we believe … the strongest public 1v1 catan agent"); the only outside numbers are v1′s, whose graph checkpoints scored 23–33% against it at 64 sims (48 games each). nobody has shown the recipe improves an already strong network, which is our case. the divergences most likely to matter: (1) how the target scales q: our engine min-max rescales q over the root's moves at c_scale 0.1, so on a two-move decision any q lead becomes a 107-logit lead; canopy divides by the span of values its search saw, at c_scale 1, so near-ties stay near-ties. on the probe's 600 positions a canopy-shaped target (unrescaled q, c_scale 1, no temperature) overrules the start on knight-or-roll 2.2% of the time against the engine target's 18.9%, with the same agreement with the 8192-sim move (88.1 vs 87.8%) and still sharp (entropy 0.08), a difference the earlier −log score couldn't see. (2) the soft run's prior temperature is not canopy's (canopy softens only through the separate output), and it is what flattens the soft run. (3) the start: canopy's search targets were always far better than its prior; from a 95% start the search's gain is close to its own noise, and the hand-off also switches the critic's objective and starts a fresh adamw on a muon-trained net. (4) one sampled world per search against canopy's per-simulation resampling. v1 ran the canopy recipe (its puct version) on a 55%-vs-F graph net and it did improve it (55 ⟶ 70.5% raw vs F over 10 iterations), but never from a ≥ 90% start.
- **benchmark against the released canopy** (marco, 2026-10-23: "before anything else"; running since 09:55, on wandb: canopy-bench-2p/main, which shows no finals, so read the records): released canopy (cullback/canopy at `6185983`, nexus-v3 iteration 315) plays in our own harness as a `CANOPY` agent, a cpu bridge to its rust search, built from v1′s bridge. four patches, all committed under `third_party/canopy/`

with a reproducible build: v1′s two (the per-move search budget and the bridge) and two of ours. ours make the bridge fair: v1′s sent canopy our true dev cards, while ours sends counts only. on 8,510 audited positions the bytes canopy received did not change when the opponent's hidden cards were redealt (in 2p the resource hands are public anyway). they also make tree reuse exact. the rules are ours (15 vp, discard above 9, no friendly robber, random dice). canopy trained with a friendly robber and balanced dice, so those inputs are new to it, and it cannot express a one-card year of plenty (counted when offered). it reuses its tree less than it does natively: it re-roots only within its own turn and through forced moves, and about half its decisions keep visits, probably a small handicap. in the smokes there were 0 fallbacks, 0 moves our engine refused and 0 rule mismatches. canopy's 1600-sim search costs 5–7 cpu-seconds a decision, so the rows are small, enough to tell "much stronger" from "much weaker" (±12–15 pp) but not a close call:

1. canopy 1600 vs our raw policy (`puffer-graph2/public@e3600`), 60 games;
2. canopy 1600 vs our 2048-sim gumbel search on that network, 40 games;
3. canopy's raw network vs ours, 1,000 games;
4. canopy 1600 vs F, 40 games, as a common reference;
5. canopy 1600 vs the deployed 1v1 bot (`BELIEF` on `puffer-graph2/full@e3350`, 16 worlds), 40 games.

about 14 h on 4 cpu cores. the smokes (4 games a row, plumbing only) had canopy winning 4 of 4 against our raw policy, twice.

marco, rightly: "why would we play 60 games? how are we getting stat sig results with that?" 60 games is ±12.6 pp and 40 is ±15.5, so those rows could only detect a blowout. i had launched a config whose game counts were cut to fit canopy's cpu cost instead of fixing the cost. the fix being built: canopy's network on the gpu (its cuda backend), many games batched per bridge, 1600 sims kept, and every row 1,000 games on the first 500 boards of `heldout-2p` (the boards of `831eea84`). rows 1 and 2 then pair game by game. the cpu run is kept only for row 1′s 60-game early read.

- **result, raw network vs raw network** (1,000 games, cpu, 8 minutes): canopy's network without search scores **30.5** 27.7–33.4 against our `puffer-graph2/public@e3600` (`1c916131`; 34% from the first seat, 27% from the second; 0 fallbacks, 15 one-card years of plenty it couldn't express). so our raw policy is the stronger network by a wide margin. two things lean against canopy here. its raw policy reads the opponent's unplayed dev cards as an expectation, where self-play encoded the exact hand (its search deals hands, its root input doesn't). and it never saw our dice or robber rules. its 1600-sim search is what the other rows test.
- **row 1′s early read** (60 games, cpu, 3 h 9 min; an underpowered row, kept only as a blowout check): canopy 1600 scores **45.0** 33.1–57.5 against our raw policy (`00291a24`; 40% from the first seat, 50% from the second; 0 fallbacks, 0 refused moves). so neither side is crushing the other, and nothing finer can be read from 60 games. the cpu run was stopped at 13:06 once row 1 landed.
- **the gpu build** (done 12:32, on branch `wt/canopy-gpu`, not merged yet): canopy's cuda backend, the same commit, patches and checkpoint. on 300 positions at 1600 sims it picks the same move as the cpu build 300 of 300 times (root values differ by 2.5e-4 on average, about 20 times less than changing the search seed), and its outputs don't depend on how many games are batched together. with 4 bridges of 256 games each it does about 5.3 searched decisions a second while sharing the gpu with the critic refit, against 0.2 for one cpu bridge. that puts a 1,000-game row at roughly 4–5 h (rows 1, 4, 5) and 7–9 h for row 2, whose opponent also searches. the cuda binary can't be rebuilt byte for byte (a hash map in the kernel compiler), so the installed copy is the one the records name.
- **canopy's search vs our search, launched first** (13:41, on the gpu; on wandb: canopy-search-2p-gpu/main, no finals, read record `96d94fab`): marco asked for this row soonest, so it runs on its own ahead of the rest of the gpu config, which is still under review. the cuda build was merged on its own (`b385ea9`; code ids unchanged) and the critic refit paused at e266 to free the gpu (resumable). canopy 1600 vs our raw public network in the 2048-sim gumbel search of `831eea84` (the spec resolves identically), 1,000 games on heldout-2p's first 500 boards in both seat orders: ±3.1 pp, about 80% power for a 4–5 pp gap from 50%. it pairs game by game with row 1 (canopy vs our raw policy) when that runs, which says what our search buys against canopy.
- **the rest of the gpu rows, reviewed and ready** (`configs/experiments/canopy-bench-2p-gpu.yaml`, merged into integration; launches when row 2 ends, since main is the runner): row 1 vs our raw policy, row 5

vs the deployed 1v1 bot, row 4 vs F, then row 2, which it finds already played (same record id). 1,000 games each, about 19 h on a free gpu, with the critic refit still paused (the two don't fit in 8 gb together). what each comparison can detect at 80% power (two-sided 5%): row 1 alone vs 50%, 4.4 pp; rows 1 vs 2 paired (what our search buys against canopy), 5.2 pp; row 1 vs the raw-vs-raw `1c916131` (what canopy's search buys), 5 pp; row 4 vs our policy's 95.6 vs f on the same boards, 2.2 pp. the review also made the cuda build installable only as the pinned binary, added a behavioural fingerprint of it (64 fixed positions), and extended the fairness audit to redealing victory point cards. gpu and cpu canopy are the same player statistically but not bit for bit over whole games (29 of 40 smoke games identical, 2 winners changed), so their records key apart.

- ☐ **refit the critic after ppo, then search with it** (marco, 2026-10-23): during marl ppo the critic chases a policy that keeps changing, so it probably never converges on the final one; freeze the policy, keep training the critic on many of the final policy's games to predict the outcome, and mcts should get better with the more accurate critic. there is a second reason to expect it: the start's critic is discounted (v-trace targets at $\gamma = 0.999$, $\lambda = 0.95$), so it predicts something closer to 0.999^(moves left) × p(win) than p(win), and that is not the undiscounted win/loss alphazero's value head trains on either.
  - plan: the policy stays bit-identical by splitting the shared trunk into two copies, the policy's untouched and the critic's trained with its value head on the monte-carlo outcome ($\gamma = 1$, $\lambda = 1$) of raw-policy self-play in the start's own env; a value-head-only arm on the shared trunk if it is cheap. a cpu probe first measures the start's critic against the outcome (calibration by game phase, the discount, and how much a refit gains on held-out games). judged in the search harness, with only the critic different: mcts with the refit network vs the raw start at 512 sims on the 1,000 select games (paired with `856a0421`, the start's 57.4) and at 2048 sims on `heldout-2p` (paired with `831eea84`, 62.4). built and merged 2026-10-23 (`3dc0450`): `train.freeze: [actor]` takes the policy out of the optimizer (its logits stay bit-identical through training, checked at every launch against `init_from`), and `init_from.split_shared` copies the shared trunk into separate policy and critic networks that reproduce the start exactly (identical logits, values and 512-sim searches). `configs/experiments/critic-refit-2p.yaml`: arm `main` refits the whole split critic (376k parameters), arm `head` only the value head (74k); adam 3e-5 ⟶ 3e-6 over 500 epochs (400k games of the frozen policy at temperature 1, of which 63% reach the loss), 256-step segments so about two thirds of the targets are real outcomes and the rest bootstrap. a fixed set of 2,048 held-out games on boards outside the training pool (`tools/critic_heldout.py`) scores every checkpoint, paired, against two calibration-only fixes of the start; the run counts as converged if e500 is within 0.002 nats of e450.
  - **cpu probe of the start's critic** (5,120 raw-policy games on the start's own boards, 827k decisions): against the undiscounted outcome its log loss is 0.370 (0.682 for a constant guess), falling from 0.68 in the first tenth of a game to 0.11 in the last. it is discounted as expected: the two seats' values sum to 0.936 rather than 1, and v ≈ 0.999^(decisions left) × p(win) fits where v ≈ p(win) does not, so it is under-confident about wins (a predicted 0.74 wins 83% of the time). the only headroom found is calibration: dividing each seat's value by the two seats' sum, with no training at all, cuts the held-out log loss by 0.010 0.008–0.011 nats, and the best fit, a refit of the value head on the frozen trunk, by 0.012; none of them ranks positions any better (auc within the same stage of the game 0.894 for all). the gumbel root rescales q to its own range, which cancels most of a calibration change, so on this evidence alone i don't expect the refit to move the harness rows; whether a separately trained critic trunk on 400k games finds discrimination the head couldn't is what the gpu run tests. the run reports both.
  - decision rule, fixed before launch: e500 at 512 sims paired with `856a0421` and at 2048 sims paired with `831eea84`, mcnemar with holm at 0.05 over the two; at 1.6–1.8 pp paired se, 80% power needs an effect of 5 pp, so a null reads "no effect of 5 pp or more". if a main row passes, the head arm runs and main vs head (paired) tests whether the gain comes from a more accurate critic or only from a rescaled one. 8–14 gpu hours for the main arm, not yet measured with two trunks. the main arm launched 2026-10-23 09:57, alongside the canopy rows (on wandb: critic-refit-2p/main).
- ☐ **monte carlo playouts instead of a tree** (marco, 2026-10-23): for the policy's top k actions, play n full games from each with the raw policy on sampled determinizations and take the action with the best win rate (k ≈ 5, n ≈ 1,000). no critic at all, and since the policy plays fair (it sees only its own observation), a playout in a world dealt from the tracker is a proper sample of the outcome, so this is one step of policy

iteration (tesauro's backgammon rollouts). the engine has the pieces: `cpp.sample_world` deals a world from a seat's tracker and games copy.

- cost, from our own numbers: a 2p game is 159 decisions (81 of them the searching seat's, `831eea84`), so a decision is 80 decisions from the end, and k = 5, n = 1,000 is 400k policy steps a decision. ppo's rollouts run at 42k decisions/s on the 2070 super, so 10 s a decision, 10 min a game, and a 1,000-game harness row 150 gpu hours against mcts's 3.9 h at 2048 sims; on the bot vm a decision would take minutes. so not as a player at that size. feasible as (a) a reference: q^π(s, a) for the top actions at a few hundred positions (500 positions ≈ 1.5 gpu hours), with common random numbers (the same worlds and dice for every action), ±1.6 pp per action at n = 1,000, which would grade the search's choices and the critic far better than one noisy 8192-sim search; and (b) a cheaper player with sequential halving over the actions or truncated playouts (d moves, then the critic), at maybe 5–10× the cost of mcts at 2048. not queued.

- ☑️ **training one model to train for up to `n` players and arbitrary maps** (should be possible by using masked values for numeric features and GNN already handles arbitrary hex boards)
  - the harness already does the masking: the row is padded to the env's maxima with a validity bit per slot, absent seats are all-zero, the global block one-hots the number of seats, and the per-seat value head predicts every present seat. `env.scenarios` draws each game from a weighted list of (player count, rules, map generator).
  - **running since 2026-10-17 02:14** (live on wandb: ppo-mixed/main, re-synced every 20 min): `configs/experiments/ppo-mixed.yaml`, `ppo-4p`'s recipe over six equally weighted scenarios: 2, 3 and 4 players on BASE and on `hexagon` boards (BASE shape, shuffled numbers, generated harbors), same decisions per epoch as `ppo-4p` (3800 epochs, about 10 h). blob-shaped boards of 15–30 tiles (`random-4p`) stay out of training because they would double the row (6316 floats vs 3202) and the memory; the run is evaluated zero-shot on them (`heldout-random-4p`). finals: vs F on `heldout-2p`, `heldout-4p`, `heldout-mixed`, `heldout-random-4p`, and head to head with the specialists (`ppo-4p` on 4p boards, `puffer-graph2`'s belief policy on 2p boards). the question is whether one network matches the specialists at equal decisions; if the blob number is poor, the follow-up trains on blobs with fewer games in flight; 5–6 players (the extension map) comes after, since the engine lacks the special building phase.
  - **result** (finished 2026-10-17 13:54 after 11.5 h; best `e3550` by `select-mixed`, 83.6; 1B agent steps = 249M decisions, the same as `ppo-4p`). score share vs F: 2p 92.3 91.6–93.0 (`b2690265`, n=5,000; the 2p specialist has 95.2), 3p 81.1 79.0–83.0 (n=1,500, parity 33), 4p 77.0 75.7–78.3 (`0f448f68`, n=4,000; the 4p specialist has 78.6). zero-shot on blob maps it never saw (`heldout-random-4p`, 15–30 tiles): 73.9 72.5–75.3 (`7c17d0ae`, n=4,000). the mixed suite as a whole: 79.8 (`9b470a17`, n=4,500).
  - head to head it loses to both specialists: 45.7 44.4–47.1 vs `puffer-graph2`'s belief-averaged policy in 2p (`340ce1db`, n=5,000), and a 39.3 37.8–40.9 share vs `ppo-4p`'s 53.0 with two F at the table in 4p (`32aab9f3`, n=4,000). **correction (2026-10-18):** the 4p number is not evidence. two identical copies of `ppo-4p` at the same table split 40.45 / 52.75 (`b07e5bd9`), because slot 0 always moves right after the F in slot 3 and F robs the seat after it (the tracker item below has the diagnosis). the swapped order (`configs/experiments/slot-order.yaml`, `0903fae0`, n=4,000, run 2026-10-18: the specialist 42.65 in slot 0, the generalist 50.85 in slot 1) settles it. averaged over both orders the generalist's share is 45.1 against the specialist's 47.8, a paired −2.7 pp −4.7–−0.8 per game, so the 4p specialist does win, by three points rather than fourteen.
  - reading: one network handles any seat count and board shape within 2–3 points of a specialist against F, and the graph net transfers to board shapes it never trained on nearly for free (73.9 on blobs vs 77.0 on BASE), but every setting got a third of the decisions, the specialists win the direct matches, by four points in 2p and three in 4p. so this is a capacity-vs-budget question: (my suggestion, not queued) either give the generalist the specialists' per-setting budget (3× the decisions, about 35 h) or fine-tune it per setting for a few hundred epochs; if the gap closes, the generalist is the model to ship.
- ☑️ **train the generalist on blob maps too**, with fewer games in flight to fit the 6316-float rows, now that zero-shot says the architecture copes; the test is `heldout-random-4p` against the 73.9 above.
  - **running since 2026-10-20 14:51** (live on wandb: ppo-mixed-blob/main): `configs/experiments/ppo-mixed-blob.yaml`, `ppo-mixed` with `random-4p` as a seventh scenario of equal weight, the same agent steps and optimizer steps (512 games in flight over 7,600 epochs instead of 1,024 over 3,800, the same 32,768-row minibatch), and the lambda critic default. the blobs set the dims (30 tiles, 122 nodes, 151 edges, 611 actions against 19 / 54 / 72 / 352), so every row is padded to them: memory is not the limit (5.5 GiB peak), compute

is: 11.5k agent steps/s against `ppo-mixed`'s 24.7k, about 24 h for the full budget. the padding is the cost: six of seven games are 19-tile boards run at blob size.
  - **result** (finished 2026-10-21 15:02 after 24.2 h; best `e7600`, the last checkpoint, by `select-mixed`, 78.8, still climbing at the end: 76.8 at e7300). it lost on the blobs it trained on: 67.4 65.9–68.8 vs F on `heldout-random-4p` (`75cab037`, n=4,000) against `ppo-mixed`'s zero-shot 73.9 72.5–75.3 (`7c17d0ae`, same suite and lineup). and it lost everywhere else too: 2p 90.1 89.3–90.9 (`799eac40`, n=5,000; `ppo-mixed` 92.3), 4p 70.6 69.2–72.0 (`ac020b20`, n=4,000; 77.0), the mixed suite 75.0 73.7–76.2 (`fb5a66f8`, n=4,500; 79.8).
  - reading: a drop of 2–7 points in every setting, the added one included, is not the seventh scenario taking a seventh of the decisions instead of a sixth; it is something that changed for all of them. three things did, none tested on its own: the lambda critic default (`ppo-mixed` trained with the older v-trace critic), half as many games in flight per epoch (512 over 7,600 epochs instead of 1,024 over 3,800), and every row padded to blob size. for an island table `ppo-mixed` is the stronger bot until one of those is isolated.
- ☐ **5–6 players** (the extension map): needs the special building phase in the engine first; after that it is one more scenario in `ppo-mixed`.
- ☐ **adding trading to the engine**
  - the rules side is done: cppanatron implements domestic trades (offer, accept, reject, confirm, cancel) in parity with catanatron. what is missing is everything in front of it: the map-agnostic action space (`GraphActionSpace`) leaves domestic trades out, the env never offers them, and the encoder has no trade-negotiation prompt. so "adding trading" means an action-space extension plus the env, the mask and the trade prompts in the row.
- ☐ **allowing the model to "learn trading"**
  - do not increase action space explosively, either
  - have the model predict the player trade action and a float for each of the 5 types of elements, for both what they give and what they get (11 extra options in the action space), and round to the nearest integer for each of the 10 floats, or
  - have the model predict the player trade action and a float for each of the 18 possible integer choices for each of the 5 types of elements, for both what they give and what they get (181 extra options)
  - i suspect that training while allowing this from scratch will likely not result in the model learning to trade well. instead, we should first train a competent model using the ppo recipe, then take random game states, try applying all every possible legal trades to the current state, and evaluate which trades modify the game state such that the new game state is better for both sides of the trade, according to the critic model
  - plan, in that order: (1) the critic-side analysis first, because it needs no action-space change: sample positions from the trained 2p and 4p networks' self-play, enumerate every 1-for-1 and 2-for-1 trade between the seat to move and each opponent, apply it to a clone of the position and read both seats' win probabilities from the critic; report how often a trade raises both, by how much, and what the typical trade is. that says whether trading is worth an action space at all under this critic. (2) if it is, the compact head (option one above) for the proposer and a yes/no for the responder, trained from the ppo checkpoint with the trade actions masked in at a low entropy bonus.
  - (1) done 2026-10-17: `tools/trade_oracle.py`, results in `report/data/trade-oracle-{ppo-4p-main,puffer-graph2-full}.json`. 3,000 positions each from the network's own self-play (temperature 1, about 3.4 positions per game, 884 / 916 games), every 1:1, 2:1 and 1:2 both hands can pay (about 73 per 4p position), applied through the engine's own offer/accept/confirm path, both seats' win probabilities read from the acting row (the only row shape the critic was trained on; a first version also read the partner's own row and a review showed that to be off-distribution noise). the critic's own noise floor is reported next to every number: its per-seat values don't conserve under a trade (median residual 1.1 pp in 4p, 0.5 pp in 2p), so the 0.5 pp and 1 pp rows are within noise and the 2 pp row is the one to read.
    - 4p (`ppo-4p/main@best`, private): a trade that raises *both* seats' win probability by ≥ 2 pp exists in 6.2% of positions [5.4, 7.2]; by ≥ 1 pp in 25.8% [24.2, 27.5]; by ≥ 5 pp in 0.4%. the best mutual trade gains the mover 1.2 pp and the partner 0.9 pp on average, and costs the two seats not in it 1.1 pp. 92% of mutual trades are 2:1 (the mover gives two; at the private level the mover's row sees the partner's hand only as a size, so this is partly what the critic can see). the mover's selfish best trade gains it 7.8 pp and costs the partner 3.7 pp on average: trades are mostly zero-sum, as expected. most common mutual trade: two wood for an ore.

- 2p (`puffer-graph2/full@best`, full information): ≥ 2 pp in 0.3% of positions [0.1, 0.5], ≥ 1 pp in 4.3%. two-player catan is nearly zero-sum for this critic, so trading would be an action space that is almost never used.
- so: worth an action space in 4p (a modest, real pool of 2:1 trades the critic likes for both sides, each costing the other two seats), not in 2p. step (2) waits for the mixed run and your call on the head (option one above is the default).
- **2026-10-19, (1) rerun on the lambda critic** (`aux-hidden-4p/plain@e600`, the first net whose four per-seat win probabilities sum to the realized 1.00; `report/data/trade-oracle-aux-hidden-4p-plain.json`, 3,000 positions from 846 self-play games, 229k trades, 3 min). the calibrated sum does not buy a cleaner oracle: the critic's residual under a trade is the same size as before (median 1.0 pp against 1.1, p95 3.8 against 4.8) and still reaches 2 pp in 90% of positions, and the mutual pool shrinks: a trade that raises both seats by ≥ 2 pp exists in **2.5%** of positions [2.0, 3.2] against the v-trace critic's 6.2% (whose sum sat at 0.93 and moved more under a hand change), by ≥ 1 pp in 22.7% [21.2, 24.3], by ≥ 5 pp in none. the best mutual trade gains the mover 0.8 pp and the partner 0.7 pp at the median; 94% of mutual trades are 2:1 (two sheep for a wheat, two wood for a wheat, two wood for a sheep lead the list), the other two seats lose 1.1 pp on average, and the mover's selfish best trade (+7.2 pp for it, −3.3 for the partner) is mutual in 0.1% of positions. the independence reference says what the 2.5% means: if a trade's effect on the partner were unrelated to its effect on the mover, 77% of positions would show a mutual one; so under this critic trades are nearly zero-sum between the two parties, and the mutual pool sits at the critic's own noise floor. two readings follow: a trade head would have about one position in forty with a trade worth ≥ 2 pp to both, a signal the critic itself resolves at the margin; and any harness that proposes trades by this critic's reading proposes gains the size of its noise. marco's proposal this evening, the trade decision outside the action space as a critic-driven harness (enumerate, apply on a determinized clone, read both seats from the mover's row, propose the best mutual trade; responders judge offers the same way), is being designed against this; its expected edge in play is bounded by these numbers, and resolving a one-point edge takes about 15,000 paired games.
- **2026-10-20, (1b) the harness, built and piloted.** marco's rule set, built overnight in a worktree (three phases of parallel agents, an adversarial review, 543 tests): a `TRADE` agent key that plays the policy everywhere and, once per own turn after the roll, enumerates every 1:1, 2:1 and 1:2 bundle it can pay, reads the critic once for all of them (about 84 rows a read), keeps the bundles whose ego-view gain is positive for it and for the partner, drops the pareto-dominated ones and offers the rest in order of its own gain, up to three a turn; a responder accepts iff its own mean gain over eight worlds is positive, and the proposer confirms the acceptee best for itself. trades stay outside the action space: the engine got one side channel (`VecEnv.set_action`), and the row layout, the checkpoints and training are untouched; fairness is black-box (the choices are identical on any game with the same public state, `tests/test_trade_fairness.py`). native F never accepts an offer (its one-ply value sees no cards move on an accept, and the strict comparison keeps reject), so every table is the same network against itself with the trading seat as the only asymmetry. the pilot (`trade-harness-pilot`, 2026-10-20 03:44, four rows of 200 games in five minutes): the proposing seat against three responder-only copies takes **46.5** 39.7–53.4 of the wins against a parity of 25 (`0fc4352d`; 14 offers and 12 confirmed trades a game, 85% of offers land, cancels under 1%), **55.5** 48.6–62.2 when it re-evaluates after every build (`1d253765`, 26 offers and 22 trades a game), **31.0** 25.0–37.7 at a 1 pp margin (`c34e389c`, 4 offers a game), and **46.0** 39.2–52.9 reading the true hands (`09893ca0`, unfair, cited as the ceiling), so the determinization costs nothing. the bundles are the oracle's, mostly 1:1 and 2:1 with sheep for wood and ore for brick on top, and the gains the critic claims for them are the oracle's too: +1.35 pp for the mover per offer (median 1.2), +0.4 for the partner, +0.6 on an accepted read, −0.3 on a rejected one (`tools/trade_pilot.py`). so the claimed gains sit at the critic's noise floor and the realized effect is the largest in this report: a dozen small trades a game compound into twenty points of share, paid by the seats not in each trade (the responders' shares 17–19). the 4,000-game rows (`trade-harness`, 03:50–04:24, 34 min for 16,000 games) confirm it: m1, the proposing seat against three responder-only copies, **45.5** 43.9–47.0 (`ff0c2cb2`; 14.7 offers and 12.4 confirmed trades a game, 84% of offers land, the responders at 18.0 / 18.3 / 18.3) against its no-proposer control's 25.0 (`7ffdd60f`, the same games with a plain policy in the seat), paired **+20.5 pp**, 1,204 games won only with trading against 386 only without (mcnemar $p < 10^{-90}$); m2, the 1 pp

arm, **31.7** 30.3–33.2 (`d7790a27`; 3.8 offers and 3.2 trades a game) against 25.0 (`7c06e560`), paired +6.7 pp ($p < 10^{-14}$). so under this critic the margin is share thrown away: the 0 pp rule trades four times as often as the 1 pp rule and wins three times as much over parity, which says the critic's trade judgement carries signal well below its own per-read noise, and that the oracle's reading (gains the size of the noise, so a null expected) was right about the per-trade gains and wrong about what a dozen of them a game add up to. open: the symmetric table (everyone proposes), the vs-F number (needs an F that answers offers), and whether a policy trained with trades on the table keeps the edge. m3 (`trade-harness-m3`, 05:31–06:38, 8,000 games) asks whether accepting pays: at a table of three proposers, the responder-only seat takes **12.7** 11.7–13.8 (`fe7baed5`) and a seat that rejects every offer 13.2 (`58f7c074`), paired −0.45 pp (371 games won only by the acceptor, 389 only by the rejecter, p = 0.54). so accepting buys nothing: the proposer captures the surplus of each trade it picks, the acceptor's own read of a gain is noise around zero, and the passive seat is crushed whether it trades or not (13 against a parity of 25, the three proposers at 27–31). the edge of this harness is in proposing. m4 (`trade-harness-m4`, 14:35–14:51), the design's row for a positive m1: the same tables with the 4p specialist `ppo-4p/main@best` playing every seat and the calibrated plain critic judging the trades: the proposing seat takes **41.2** 39.7–42.7 (`c4900f87`; 13.8 offers and 11.6 trades a game) against its never-trading control's 25.0 (`7164d396`), paired **+16.2 pp** (1,080 games against 433, $p < 10^{-60}$). so the edge holds with the strongest 4p player, four points smaller than with the weaker plain policy: a better player gets less from the same trades, or the critic of another network reads its positions slightly worse.

- ☑️ **build the best possible full-information tracker for any number of players** that resolves hidden information (e.g. a robber steal between 2 other players) if either of the other players reveals what happened in the trade via a new action
  - the only hidden information are dev cards and robberies.
  - for example: let us be player A. at the start, we know player B's hand is a wood and a sheep. player C's hand is a brick, an ore, and a wheat. if player B robs player C, and then buys a road, we can resolve that player B now has a sheep, while player C has an ore and a wheat. or if player C robbed player B, and then bought a dev card, we know that player C has a brick, and player B has a wood.
  - another example: let us be player A. we have 6 knights played, player B has 6 knights played, and player C has 1 knight played. then, if player C plays another knight, we know that there are no more knights in anyone else's hand, or in the development card deck, because player C has just played the 14th (and last) knight. we should then never account for any knights in the game. this is important because say if player B has largest army, we know we should stop trying to contend for largest army because we can't get buy any more knight cards. obviously this goes way further than this, since we should be updating our prior on our belief set for the sampling determinization method every time someone plays a dev card.
  - where the current `HandTracker` (C++, `engine/cppanatron/include/cppanatron/tracker.hpp`) stands against these: it keeps, per seat, a known lower bound per resource plus a count of unknown cards, and one global pool of unknown cards derived from the bank, so both robbery examples above should already resolve (when B pays wood + brick the brick comes out of B's unknown card, the pool is left with ore + wheat, and C's two unknown cards can only be those). the knight example is covered on the sampling side (`unseen_development_cards` = deck − my hand − every played card, so after the 14th knight no sampled world holds one) and the row carries every seat's played cards per type plus my own hand, so the network can count too. what it does *not* do: it keeps no prior over *which* card a hidden steal took (a steal from a hand of brick, brick, ore is sampled as brick or ore with equal weight instead of 2:1), it drops evidence from what seats chose not to do, and it is uniform over consistent worlds only up to those two approximations.
  - plan: pin the two examples above as tests, then an exact-posterior tracker that keeps a weighted set of hypotheses over the hidden steals (pruned by every later payment, merged when they reach the same hands, capped with a fall-back to today's pool) and samples worlds from it; measure (a) agreement with brute-force enumeration on simulated 3p/4p games and (b) `BELIEF` and `MCTS dets=K` vs the current tracker on `heldout-4p`.
  - done 2026-10-17, the tracker and (a): both robbery examples and the 14th-knight example are pinned as engine tests, and the existing pool tracker already resolves all three. the exact mode (`tracker=exact` on `BELIEF` and `MCTS`; `pool` stays the default) keeps weighted hypotheses over hidden steals, branched by the victim's count of each kind, pruned by every later payment, merged when they reach the same hands,

capped at 4096 (the most seen in 424 simulated 2–6p games: 748; 2 on average), falling back to the pool past the cap. against brute-force enumeration on 160 positions the exact belief is identical (total variation 0.0); the pool tracker's sampled deal is off by 0.12 (random play) to 0.18 (value play) total variation on average and up to 0.9 in the worst position, so the approximation is real, not just theoretical. sampling a world costs the same (about 12 µs). (b), the `heldout-4p` evals, is queued on the gpu.

- (b) is `configs/experiments/tracker-modes.yaml` (2026-10-18): the two tracker modes of `ppo-4p`'s belief policy, 16 worlds each, at one 4p table with two F, 500 boards × 4 rotations, so every game is a paired comparison. runs after `aux-hidden`. if exact beats pool there, the same pairing with `MCTS:sims=256:dets=4` follows (about ten times the cost).
- **2026-10-18, (b) result: void at this level, and it exposed an eval artifact.** the table ran in 3 minutes (2,000 native games, record `79147906`) and gave exact 40.4 38.2–42.5 against pool 52.7 50.5–54.8, a paired −12.3 pp [−16.5, −8.1], the same at every seat. that is not the tracker. a cpu replay of 1,208 decisions (diagnostic scripts in the session scratchpad, not in the repo) showed that at the `private` level every sampled world encodes to the true row (0 of 4,832 worlds per mode differ in any feature: the encoder zeroes exactly what a world could change), so `BELIEF` with either tracker is the argmax policy, and the record is game for game identical to `ppo-4p`'s own `[POLICY, POLICY, F, F]` control (`b07e5bd9`: 40.45 / 52.75). the gap is the lineup. the suite seats `lineup[(seat + r) mod 4]`, so slot 0 always moves right after the F in slot 3, and F (`enemies=next`) robs and blocks the seat after it: in the replayed games the F in slot 3 put the robber on slot 0's tiles in 280 of 280 moves, slot 0 was stolen from by an F 263 times and slot 1 39 times. with `F:enemies=all` the difference flips to about +7 pp, so even then the two learner slots are not exchangeable: a paired 4p table needs both slot orders (or the sum, as the "two copies" row in Table 2 does). this voided the 4p head to head of `ppo-mixed` against `ppo-4p` as first reported; `slot-order` ran the swapped order and the `ppo-mixed` item above has the corrected number. what survived: on those 1,208 4p decisions the exact deal reproduces the other seats' true resource hands in 72.0% of worlds, the pool deal in 60.5%, with no fall-back (1–71 hypotheses), and the exact-mode env is bit-identical to the pool-mode env. the tracker can only matter to an agent that reads the sampled hands: in 4p that is `MCTS` with dets (search plays out steals and builds with them) or a full-level 4p net played through `BELIEF` (none trained; `ppo-4p` is private, and whether 4p should train at `full` and play through `BELIEF` as 2p does is an open question), and in 2p never (no hidden steals, the two trackers coincide). box: `MCTS:sims=256:dets=4` exact vs pool with `ppo-4p`'s net, both slot orders, 200 games (about 3 h; after the critic fix is read, since search backs up the critic). queued 2026-10-19 as `configs/experiments/tracker-search.yaml` (2 × 50 boards × 4 rotations = 400 games).
- **2026-10-19, the search pairing: a tie, game for game.** `tracker-search` ran 07:11 to 07:38 (27 min for the 400 games, not 3 h: the searchers batch their leaves across the 256 games in flight): `MCTS:sims=256:dets=4` with `ppo-4p/main@e3650`'s net, exact against pool at one table with two F, 50 boards × 4 rotations in each slot order. exact 44.5 37.8–51.4 against pool 52.5 45.6–59.3 with exact in slot 0 (`89915b43`), pool 44.0 37.3–50.9 against exact 52.0 45.1–58.8 with the orders swapped (`139796c6`): 193 wins each out of 400, a paired difference of 0.0 pp [−3.2, 3.2] over the two orders (bootstrap over the 200 board × rotation pairs), and the slot after the F loses 8 points in both orders (44.3 against 52.3: the artifact above, now on the searcher). the pairing is tight because the two games of a pair are mostly the same game: on the same dice the same seat won in 186 of the 200 pairs, in the same number of turns in 169, with the same number of decisions per seat in 161, so at 4 worlds × 256 sims the sampled hands rarely differ enough to change the search's choice; the 14 pairs that did change hands split 6 to pool, 4 to exact, and 4 where exact won one game and an F the other. verdict for the item: the exact posterior is right (identical to enumeration, where the pool deal is off by 0.12–0.18 total variation) and costs nothing, but no agent in this repo plays differently for it: the private-level policies cannot see it, and search at the sim counts we can afford does not act on it. it stays as the default-off `tracker=exact` option; what would read it is a full-level 4p net played through `BELIEF` (the open question above) or search with many more worlds, neither queued.

- ☑️ **train the model to predict the full-information observation from the public-information observation** instead of sampling from belief
  - the natural target already exists: the `full` row of the same position (the env can encode any level), so this is an auxiliary head on the shared trunk predicting the hidden features (opponents' resource and dev-card counts) with a masked regression loss, trained alongside PPO at the `private` level. the test is whether the policy's score moves, not the auxiliary loss. after the mixed run.

- implemented 2026-10-17 (`model.args.aux`, `train.aux_coef`, experiment `configs/experiments/aux-hidden.yaml`). the hidden set is exactly the 15 per-seat features the encoder zeroes for other seats below the level that shows them (5 resource ratios, actual VPs, 5 dev-card counts, 4 playable flags), pinned against the encoder by a test; the targets come from the `full` players block the row already carries for a critic at another level (`critic_level: full`), which the shared trunk never reads (a test perturbs that block and checks logits, values and the predictions are bit-identical), so the policy stays fair and evals encode only the `private` row. loss: masked MSE in the encoder's feature scale over present opponent seats on acting rows, coefficient 0.3 from the measured gradient ratio against ppo's loss. with the coefficient at 0 the training is bitwise the plain recipe. the run is the arch ablation's `graph-s` cell plus the head (100M steps, 2p, private), judged vs F and head to head against that cell on `heldout-2p`; in the gpu queue behind the ablation. per-feature R^2 is logged (`aux/*`) so the report can show what the head actually infers (hands vs dev cards).
- **2026-10-18, result:** the head does not move the policy. the aux cell scores 89.1 88.2–89.9 vs F against the plain cell's 90.7 89.9–91.5, and 47.7 46.3–49.1 head to head against it on 5,000 games (records `0e28d5cf`, `c6b7f682`; the row is in Table 1). same throughput (17.1k vs 17.6k steps/s, 1.7 h), same critic at the end (explained variance 0.930 vs 0.928), same entropy, so at coefficient 0.3 the loss acted as a regulariser that cost about a point and a half and nothing else. what the head learned is the useful part (R^2 per feature over the last 50 epochs, from the logged `aux/mse_*` and `aux/var_*`): the opponent's hand is nearly determined by the public row already, resources 0.90–0.99 (wheat 0.99, ore 0.96, sheep 0.96, brick 0.93, wood 0.90) and the actual vp count 0.99; in 2p the tracker's known cards, the hand size and the production history leave little to infer. dev cards in hand are the genuinely hidden part and stay hidden: victory point cards 0.85 (a card bought and never played is a vp card, the net works that out), knights 0.60, monopoly / road building / year of plenty 0.10–0.22, the playable flags the same. so in 2p the trunk had nothing new to encode, which is the honest reason the policy did not move. two things stay in the box: (i) the same cell at 4p, where steals and the robber hide more and the resource R^2 should drop (queued 2026-10-19 as `configs/experiments/aux-hidden-4p.yaml`: `ppo-4p`'s recipe at 750 epochs, matched to the 2p cell in decisions, as a two-arm sweep, plain and aux, with the head to head in both slot orders); (ii) the literal reading of the item, act on the prediction instead of sampling worlds: run the full-information policy on the private row with the 15 opponent features filled in by the aux head (`PREDICT`, eval only, minutes of gpu) and pair it against `BELIEF:worlds=16` of the same checkpoint. (ii) is cheap and is the next worktree.
- **2026-10-18, (ii) built and measured.** `PREDICT:ckpt=<full policy>:predictor=<aux checkpoint>` (eval only; `configs/experiments/predict-vs-belief.yaml`): the full-information network `puffer-graph2/full@best` acts on the true game's private row with the opponent's 15 hidden features overwritten by `aux-hidden/main@best`'s prediction, two forward passes where `BELIEF` runs sixteen. fair by construction, pinned by tests: the private row plus the fill is bitwise the full row, and with the true features in the head's place the agent reproduces the full-information policy's actions exactly. on `heldout-2p`, 5,000 games a row, 18 minutes for all four: 94.7 94.1–95.3 vs F (`b74d240b`; `BELIEF` 95.1, the oracle 95.2), 50.4 49.1–51.8 head to head against `BELIEF` 16 worlds (`e7e34d28`), and 49.9 48.5–51.3 with the predictions snapped to the encoder's grid (`a939fa5f`). so in 2p one predicted row is worth sixteen sampled ones, which fits the R^2 table above: what the predictor gets right (resources, actual vps) is what the tracker's worlds agree on anyway, and what it cannot know (dev cards) averages out either way. the fourth row, 63.1 61.8–64.4 against the private-level `arch-ablation/graph-s` (`751509ea`), is confounded by budget (100M steps against `puffer-graph2`'s 500M) and says nothing about the fill by itself. still boxed: the same measurement in 4p needs a full-level 4p net and a 4p predictor (neither trained), and a predictor head on the full-level network's own trunk would make it one network again.
- **2026-10-19, (i) measured at 4p: the head does not move the policy there either.** `aux-hidden-4p` ran 10:15 to 14:02: `ppo-4p`'s recipe (`private` level, 512 games in flight) at 750 epochs, 196.6M agent steps = 49.2M decisions, the 2p cells' budget per decision, as two arms, plain and aux (coefficient 0.3, the critic at the `full` level as in 2p), 1.85 h and 1.92 h at 30.5k and 29.6k steps/s, select-4p every 50 epochs (plain best e600 at 65.8, aux best e700 at 68.5, n=800; the 4,000-game finals put the same checkpoints at 64.0 and 63.2, so the best of sixteen noisy selects is 2–5 points optimistic, as a maximum of noisy points is). against three F: plain **64.0** 62.5–65.5 (`d97f69b3`), aux 63.2 61.6–64.6 (`da4cb5be`); head to head at one table with two F in both slot orders (`27a774b6`, `2eb6a290`, 8,000 games): 42.5 against 42.5, a paired difference of 0.0 pp [−2.0, 1.9] (and the slot after the F loses 9 points in both orders, 38.0 against 47.0: the artifact of the tracker item, for the

third time). what changed from 2p is what the head can know (R^2 over the last 50 epochs): the hands are now genuinely hidden, resources 0.48–0.59 (wheat 0.59, ore 0.57, wood 0.55, sheep 0.52, brick 0.48) against 0.90–0.99 in 2p, the actual vp count 0.95; dev cards as before, vp cards 0.65, knights 0.51 (0.70 at epoch 100, falling to 0.53 by the end), the rest 0.07–0.21. so in 4p steals and the robber do hide half the variance of the other hands from the public row, the head recovers that half, and the policy still gains nothing from the trunk encoding it. the box closes on the regulariser reading in both player counts; what paid was (ii), the prediction as a substitute for sampled worlds. the two arms are also the first from-scratch runs under the lambda critic default; that reading is in Section 2.

  - **2026-10-19, (iii) the head as a probe of the lstm's memory (marco's test).** `lstm-aux-4p` puts the same head on the recurrent net's memory-injected embeddings: its R^2 per feature is the markov head's or a little below (resources 0.47–0.55 against 0.48–0.59, actual vp 0.95 against 0.95, knights 0.51 against 0.51), so memory over the seat's own history adds no hand information beyond the tracker row at 4p, and the aux-shaped memory does not move the policy (paired +1.5 pp [−0.4, 3.5] against `lstm-4p`). the detail is under the markov item below.

- ☐ **define different scopes for how "markov" a model decision is**
  - right now, we only feed the model the latest view of the game state as its input
  - we could instead train a sequential model on the entire game state history, e.g. transformer or lstm
  - this would allow the model to learn things like "if someone didn't play a knight 2 moves ago when one of their high production tiles was blocked, then they probably don't have a knight among their dev cards"
  - this uses many orders of magnitude more compute, but would be only way to get "perfect" bot
  - first step is the LSTM arm of the arch ablation (a recurrent state over the seat's own row history is the cheap end of this scope); a transformer over the whole history is the expensive end and only makes sense if the LSTM beats the markov graph net at `private` level. the lstm class exists since 2026-10-17 (section 1); its 100M-step cell is in the ablation queue.
  - **2026-10-18, the lstm arm:** it ties the markov graph net head to head, 51.0 49.7–52.4 on 5,000 games, and against F (90.3 vs 90.7), at `private` level where there is hidden information to remember (Table 1). so at 100M steps in 2p, memory over the seat's own row history buys nothing measurable: the row already carries played cards, hand sizes and the robber history the net needs. the transformer end of this scope is parked unless the lstm shows a gap at the full budget or in 4p, where more is hidden; the cheap next check is the lstm at 4p against `ppo-4p`'s recipe.
  - **2026-10-19, the lstm at 4p: memory buys four points head to head.** `configs/experiments/lstm-4p.yaml` ran 14:18–16:58 (2.53 h of training and selects at 22.3k steps/s, 27% slower than `graph-s`'s 30.5k; peak 5.2 GiB): the `lstm` preset on `ppo-4p`'s recipe at `aux-hidden-4p`'s budget (750 epochs, 49.2M decisions, the lambda critic), paired against `aux-hidden-4p/plain@best`, the markov `graph-s` cell of the same recipe, budget, seed and critic. against three F: **65.5** 64.0–67.0 (`9198cef1`) against the plain's 64.0 62.5–65.5, 1.5 points, inside the noise; and the two select curves sit within one half-width of each other at all fifteen points (the lstm's best is its last, e750 at 65.2, and it was still climbing: 62.3, 61.8, 64.4, 65.2 from e600, where the plain's has been flat since e550). head to head at one table with two F in both slot orders (`5efc75ba`, `cf8368f4`, 8,000 games): **45.2 against 40.8**, a paired difference of +4.4 pp [2.4, 6.3]. the slot after the F costs about 10 points in both orders (38.0 against 48.0 by slot, the tracker item's artifact for the fourth time); the lstm pays it in one order (39.7 against 45.3) and the plain in the other (36.3 against 50.7), and the average over the two is the paired reading. so what bought nothing in 2p (51.0) buys four points at 4p against the same net without memory, while the F,F,F row and the select curve do not separate them: the gain shows against the markov net, not against F. one seed. the lambda critic calibrates on the recurrent net as on the plain one (seat sum 1.59 at e21, on 1.00 by e250, mean 1.000 over e300–750). the transformer end of this scope is no longer parked by the data; what the memory is reading is the next question, below.
  - **2026-10-19, marco's test: does the memory read hands? no.** the lstm's expected contribution at 4p is inferring the other hands from activity: what a seat built, what it chose not to build, who stole from whom, which the row's tracker (a card counter) cannot encode. the gnn + lstm would do this in its internal representation, if at all, so test it explicitly with the aux item's hidden-feature head: `configs/experiments/lstm-aux-4p.yaml` ran 17:00–19:41 (2.58 h at 21.8k steps/s, peak 5.4 GiB), `lstm-4p` plus the head (coefficient 0.3, the `full` players block as the target) on the memory-injected embeddings, so its per-feature R^2 against `aux-hidden-4p/aux`'s markov head says how much hand information the memory adds beyond the tracker row. the answer is none: over the last 50 epochs the resources are 0.47–0.55 with memory against 0.48–

0.59 without (wood 0.47 vs 0.55, wheat 0.55 vs 0.59, ore 0.52 vs 0.57, brick and sheep equal), the actual vp count 0.95 against 0.95, vp cards 0.63 against 0.65, knights 0.51 against 0.51, the playable knight 0.42 against 0.42; the only features the memory lifts are road building, 0.26 against 0.20 (playable 0.24 against 0.19), and the head's own loss is higher with memory (0.020 against 0.017). the same picture at matched epochs 100, 300 and 500, so it is not a matter of training time. the policy: **66.2** 64.7–67.6 against three F (`722b7168`; `lstm-4p`'s 65.5), select best e750 at 68.4 (the best-of-sixteen optimism again); head to head against `lstm-4p/main@best` at one table with two F in both slot orders (`561c7ee2`, `f691b7fc`, 8,000 games) 43.8 against 42.3, a paired +1.5 pp [−0.4, 3.5]: the aux-shaped memory does not move the policy either, as the head did not move the markov policy. so the four points the memory bought against the markov net (above) are not read off the hands: whatever the recurrent state carries, the hand regression cannot extract more of the other seats' cards from it than from the tracker row. one coefficient (0.3, not re-measured for the lstm); the recurrent aux path needed building for this (`tests/test_lstm_aux.py`, config schema 4). what the memory does carry is open: the order of events, or who did what to whom, are the candidates the hand features do not cover.

- ☐ **turn this into an LLM RL env/eval** (e.g. harbor task) and RL a small language model on it, and eval frontier models on how well they play catan
  - `catanllm` (the CS 159 project) already benchmarks LLM players through openrouter against catanatron bots with a few prompting strategies; v2′s game server (protocol 5: committed seeds, signed moves, verifiable transcripts) can seat an LLM client against our bots fairly. an eval is cheap (api cost, no gpu); RL-ing a small model is not on this box.
  - the eval half is built (2026-10-17): an `LLM` agent spec (`LLM:model=<openrouter id>:strategy=plan|basic|strategy|search|rules:thinking=true`) that plays through the catanatron harness next to `POLICY` and F, with `catanllm`'s five prompting strategies ported over. the prompt is public information only (opponents as hand sizes, dev-card counts, played cards and visible points; the original renderer leaked every seat's dev cards, which a test now pins), a stub model plays the same games for free so the whole thing is tested without an api key, and real-model records are marked non-replayable so a report refresh can't quietly spend money. `configs/experiments/llm-eval.yaml` is a 16-cell sweep (four models × plan/basic × vs F / vs `puffer-graph2`′s policy, 20 boards both seats) with placeholder model ids. `catanrl llm-cost` prices it from stub games: a 2p game to 15 points prompts the model about 20 times at 2–8k tokens a prompt, so a 50-game cell is about $16–26 with gpt-5 or sonnet-4.5 prices as a lower bound, and a model that actually plays well lasts longer and is prompted 70–90 times, so budget 3–4× that. waiting on an api key and a budget; the RL half is not on this box.
- ☐ **turn this task of making a strong catan bot an LLM RL env/eval**, and there is a good chance the next frontier models will become amazing at creating better catan bots.

4 log

- **2026-10-14.** `puffer-graph2/full` finished (7.95 h): 95.16% vs F on `heldout-2p`, best checkpoint e3350 by `select-2p`. `ppo-4p/main` launched right after.
- **2026-10-15.** `ppo-4p/main` finished (9.3 h): 78.55% score share vs three F, 100% vs three R. both runs backfilled to wandb. the queue's third job (`puffer-graph2/public`) never started.
- **2026-10-17.** this write-up. launched `puffer-graph2/full`'s missing search finals (MCTS 1024 and 4096 sims, one determinization, vs the belief-averaged policy) and `ppo-mixed` (2–4 players, BASE-shaped boards); the two shared the gpu badly (training at half speed, search at zero finished games in ten minutes), so the finals were stopped and go back in the queue behind the training run: one gpu job at a time from now on. started worktrees for the CNN port, the LSTM policy, the exact tracker, the trade oracle and a wandb mirror; all five were implemented, reviewed and merged into one integration branch the same day (main takes it once the gpu is idle, because the eval provenance hashes the code on disk and a live run refuses to continue under changed code). `ppo-mixed` backfilled to wandb and re-synced every 20 minutes. later the same day: the aux head (predict the hidden features) implemented and verified in a sixth worktree; the integration branch now also mirrors every real run to wandb by default. evening: the LLM player (eval half) built and verified in a seventh worktree. `ppo-mixed` finished at 13:54 (11.5 h): 92.3 / 81.1 / 77.0 vs F in 2p / 3p / 4p, 73.9 zero-shot on blob maps, loses head to head to both specialists (Section 3). the search finals started right after.

- **2026-10-18.** the search finals finished at 01:31 (11.5 h): MCTS beats the belief-averaged policy 53.4% at 1024 sims and 55.2% at 4096. main merged with the integration branch (tracker, cnn, lstm, trade oracle, wandb mirror, aux head, llm player), engine rebuilt, `arch-ablation` launched at 01:37 (graph-s, then mlp, cnn, lstm). the repo went public on github the same morning (submodule branches pushed to `mcrco/cppanatron`, `mcrco/catanatron` and a `mcrco/PufferLib` fork). the ablation finished at 13:08: graph ≈ lstm > cnn > mlp (section 1). `aux-hidden` launched at 13:11 and finished at 14:53 (1.7 h): 89.1 vs F and 47.7 head to head against the plain cell, so the hidden-feature head is a small regulariser and nothing more in 2p (Section 3). `tracker-modes` ran 14:53–14:57: exact 40.4 / pool 52.7 is a slot-order artifact, not the tracker (diagnosed on cpu the same afternoon; the `ppo-mixed` 4p head to head has the same flaw, `slot-order` queued). the critic-fix sweep ran 14:58–16:00: lambda targets put the seat sum on the realized sum within 20 epochs in every arm, the undiscounted critic sums to 1.00, the policy unchanged (Section 2). `slot-order` ran at 16:00 (one minute, 4,000 games): over both orders the 4p specialist beats the generalist by 2.7 pp paired, not 14. `search-4p` ran 16:33–17:45 (400 games): MCTS 27.0 share against a parity of 25 in 4p. the `PREDICT` agent (act on the predicted hidden features) was built, reviewed and verified in an eighth worktree during the afternoon; main took it at 17:44 and `predict-vs-belief` ran 17:45–18:03: one predicted row ties sixteen sampled worlds (50.4 head to head, 94.7 vs F). `puffer-graph2/public` launched at 18:03 (8 h of training, then its finals).
- **2026-10-19.** the critic fix became the default (`critic_targets: lambda`, `critic_gamma: 1`; the older yamls pin the old values so no stored run is re-keyed), and three experiments were queued behind the public run's finals, cheapest first: `tracker-search`, `search-4p-2000`, `aux-hidden-4p`. `puffer-graph2/public` finished training at 02:06 (7.95 h, as `full`): 95.0 vs F as a plain fair policy, the same as `full`'s belief-averaged 95.1, so the hidden dev cards buy nothing in 2p (Section 2); its MCTS finals started at 02:07 and finished at 07:10 (5.0 h, half the estimate: a one-forward opponent, 256 games in flight): MCTS 54.9 at 1024 / 55.2 at 4096 over the raw `public` policy, the `full` network's edge again. `tracker-search` ran 07:11–07:38 (27 min, 400 games): exact and pool tie exactly, 193 wins each over both slot orders, because the two games of a pair are mostly the same game (Section 3). `search-4p-2000` ran 07:38–10:15 (2.6 h, 2,000 games): 26.9 share against a parity of 25, the same two-point edge as the 400-game table, now measured to ±2 (Section 2). `aux-hidden-4p` ran 10:15–14:02 (two arms of 1.9 h, then the finals): the hidden-feature head does not move the 4p policy either, 64.0 against 63.2 vs three F and 42.5 against 42.5 head to head over both slot orders, although at 4p it recovers only half of the hands' variance (R^2 0.5–0.6 against 0.9–1.0 in 2p); the two arms are the first from-scratch runs under the lambda critic default, whose seat sum lands on the realized 1.00 by epoch 220 where `ppo-4p`'s never did (Section 3, Section 2). `lstm-4p` ran 14:18–16:58 (2.53 h), the markov item's 4p check: the lstm beats the markov `graph-s` cell head to head over both slot orders, 45.2 against 40.8, +4.4 pp [2.4, 6.3] on 8,000 games, where in 2p it tied; against three F the two are 65.5 and 64.0, inside the noise (Section 3). marco's follow-up, the hidden-feature head on the lstm's memory to measure whether the memory reads hands, needed the recurrent aux path (built, reviewed and merged in the afternoon, `tests/test_lstm_aux.py`); `lstm-aux-4p` ran 17:00–19:41 (2.58 h): the head on the memory predicts the hands no better than the markov head (resources R^2 0.47–0.55 against 0.48–0.59, the rest equal), so the memory's four points are not hand reading; the policy 66.2 vs F and +1.5 pp [−0.4, 3.5] paired against `lstm-4p` (Section 3). marco picked trading as the next item; the trade oracle was rerun on the lambda critic at 20:15 (3 min): the mutual pool is 2.5% of positions at ≥ 2 pp, at the critic's noise floor (Section 3).
- **2026-10-20.** marco's trade harness (the trading item's step (1b), Section 3) was built overnight under his rules (a calibrated critic assumed, ego-view gains, a pareto front for the proposer, +ev for the acceptor), reviewed and merged together with the second build of the night, tree reuse, leaf deduplication and a transposition table for the search (his ask for the deployed bot); the two engine branches became one engine commit and main took both at 03:40 (577 tests). the trade pilot ran 03:44–03:49: the proposing seat takes 46.5 of the wins against three responder-only copies of itself, 55.5 when it re-evaluates after every build; the 4,000-game trading rows, m3 and the three search comparison rows were queued in one chain at 03:50; the trading rows ran 03:50–04:24: +20.5 pp paired for the proposing seat at marco's rule, +6.7 pp at a 1 pp margin, the largest effect in this report (Section 3); the search comparison rows started at 04:24. the 2p row passed (no regression); both 4p rows crashed after eleven minutes at the same game and decision, an action the real game did not allow: a bug in re-rooting, found and fixed on cpu the same morning (a robber steal that has one possible card in the sampled world is a deterministic-looking edge, and re-rooting dropped the marker that makes a determinized search refuse to follow it, so the next reuse searched from the world's stolen card; engine `b596130`, with a

regression test that fails on the old engine). m3 ran 05:31–06:38 meanwhile: accepting does not pay (Section 3). the search rows reran on the fixed engines through 14:33 (correct now, no measurable effect either way); m4 ran 14:35–14:51 (the 4p specialist with trading, +16.2 pp); `ppo-mixed-blob`, the generalist on blob maps too, started at 14:51 (about 24 h). a review of the fix and of the openhex wiring the same morning found two more: with both options on, re-rooting could leave several root actions sharing one child through a transposition (the monopolies of kinds nobody holds all reach one position), so the new root counted that child once per action and the selection was skewed (a probe at 1024 sims played a monopoly that takes nothing instead of a road); engine `e9f12af` keeps the new root's children private, a no-op with transpositions off. and the openhex seat's robber move after its own discards re-rooted a tree dealt before the discards were known (three or more players, normal robber); it now searches fresh there. the 4p row with both options, rerun on the crash-fixed engine but before `e9f12af`, played through all 200 games: **25.0** 19.5–31.4 against the stored row's 28.5 on the same games (`1b01f4ac` against `90a1f06c`, paired −3.5 pp, 7 against 14 games, p = 0.19), the direction the double counting predicts; the reuse-only row (which `e9f12af` does not change) ran 11:44–12:58 on the 400 games of the stored table: **25.3** 21.2–29.7 against 27.0 (`6010fedf` against `90a1f06c`, paired −1.75 pp, 9 games won only with reuse against 16 only without, p = 0.23). so at 1024 sims and one world the retained subtree buys nothing measurable in 4p either, as in 2p. the rows with both options, rerun on `e9f12af` (12:59–14:33): 4p **25.0** 19.5–31.4 against the stored 28.5 on the same 200 games (`6601df9a`, paired −3.5 pp, 8 against 15, p = 0.21; the same share as the pre-fix row, only 6 of its 200 games changed), 2p **58.5** 51.6–65.1 against 59.5 (`aca296e2`, paired −1.0 pp, 7 against 9, p = 0.80). reading: both options are correct now and cost nothing measurable in strength, and neither buys any either at this budget; every row leans a point or three negative, none significantly, so the honest summary is a null with a slight unfavourable tilt. what they change is the shape of the search (a reused root starts with 2–3 times the nodes; the discard chain after a 7 is a lattice), not the decision at 1024 sims. the deployed bot can take them or leave them; on the cpu the transposition table adds 0.2–0.5 s to a fresh 1024-sim search.

- **2026-10-21.** `ppo-mixed-blob` finished at 15:02 (24.2 h) and lost to the generalist it was built from on every suite, blob maps included (67.4 vs 73.9 on `heldout-random-4p`), so its deploy gate failed. marco chose the alphazero setting (the 2p specialist, straight into the self-play loop, 2048-sim targets, engine work only after a positive result); the run `az-2p-2048` was built and smoke-tested on cpu during the day. the earlier alphazero sentence in Section 3 was corrected from that run's surviving transcript. `az-2p-2048` launched at 15:40 and was stopped at 18:53 after e8 on the review's rule (48.0 and 48.2 head to head against its start): one-hot search targets, not the critic (Section 3). `gumbel-edge-2p` ran 18:56–22:50: the 2048-sim gumbel search beats the raw policy it searches with 62.4 to 37.6, against puct's 55. marco: judge alphazero in the search harness, not by the raw policy, so `gumbel-edge-2p-e8` (the e8 network in the same search, paired with that row) started at 22:51.
- **2026-10-22.** `gumbel-edge-2p-e8` finished at 02:59: e8 in the search 61.8 against the start's 62.4, paired −0.6 pp −3.7–2.5 (mcnemar p = 0.75), so eight iterations of 2048-sim targets left the search where it was (Section 3). marco picked the next alphazero step (softer gumbel targets, rebuilt from the search's own statistics with the search unchanged, the scale picked on cpu against an 8192-sim reference, the rerun judged in the search harness). the island table went live on catan.marcoya.ng at 18:59 utc: `ppo-mixed@best` (73.9 zero-shot on random islands, ahead of the blob-trained run's 67.4) on a fresh 15–30 tile island in every room, "4p island vs ppo-mixed", next to the two existing tables. the bot vm turned out to run older code than the deploy plan assumed (a 2026-10-15 checkout with the bridge files copied in by hand), so the preflight and a loopback smoke ran on its own stack first: two island games, all verified, 11 ms a decision. marco played it that evening: both recorded games verified, no fallbacks, and the bots won both. the softer-target build finished in the afternoon (the trainer change, a 600-position cpu probe, three reviews): a gentler σ alone turned out to soften nothing, because the start's prior is one-hot by itself, so marco approved the probe's prior temperature of 10, and `az-2p-2048-soft` launched at 15:49 (Section 3).
- **2026-10-23.** marco asked what canopy did for alphazero and where we diverge (Section 3: its evidence is thin, and our target's q scaling and the soft run's prior temperature are the likeliest culprits), added two ideas of his (refit the critic after ppo with the policy frozen, being built; monte carlo playouts over the top actions, costed at 150 gpu hours a 1,000-game row), and asked for a benchmark of the released canopy against our 2p bot before anything else, built from v1′s canopy bridge overnight (fair hidden information and a reproducible build added) and launched at 09:55, five rows on cpu (Section 3). the critic refit was built overnight too; a cpu probe found only calibration headroom in the start's critic (0.01 nats), and its main arm launched on the gpu

at 09:57. `az-2p-2048-soft`'s e15 select lost to the raw start in the search (46.7, −10.7 pp paired, $p < 10^{-6}$) and marco stopped the run at 01:57.